

CS 229

Problems with vision and
uncertainty in ML

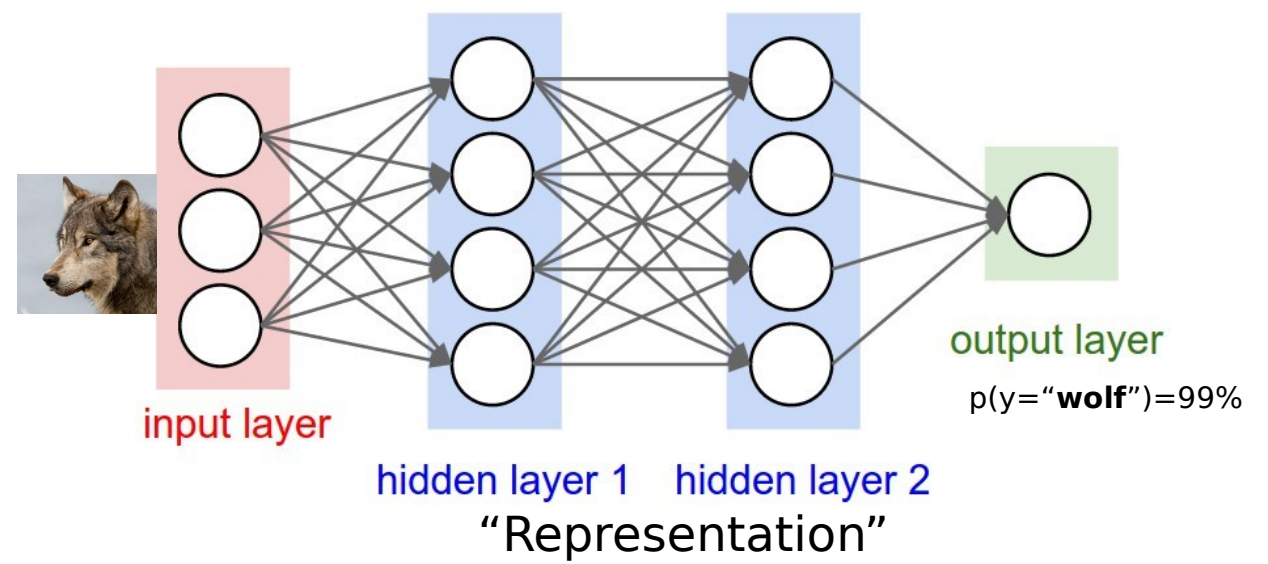
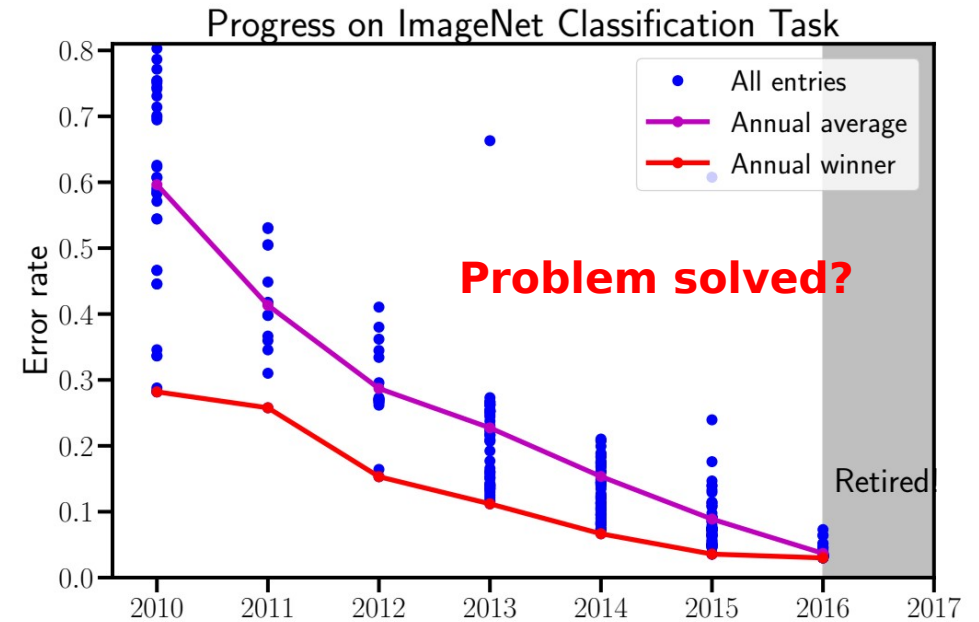


Image classification benchmarks were easy and artificial

Information used by neural nets and humans is not the same!

Hard problems remain.



Sensitivity to adversarial perturbations



Real world attacks



<https://arxiv.org/pdf/1707.08945.pdf>

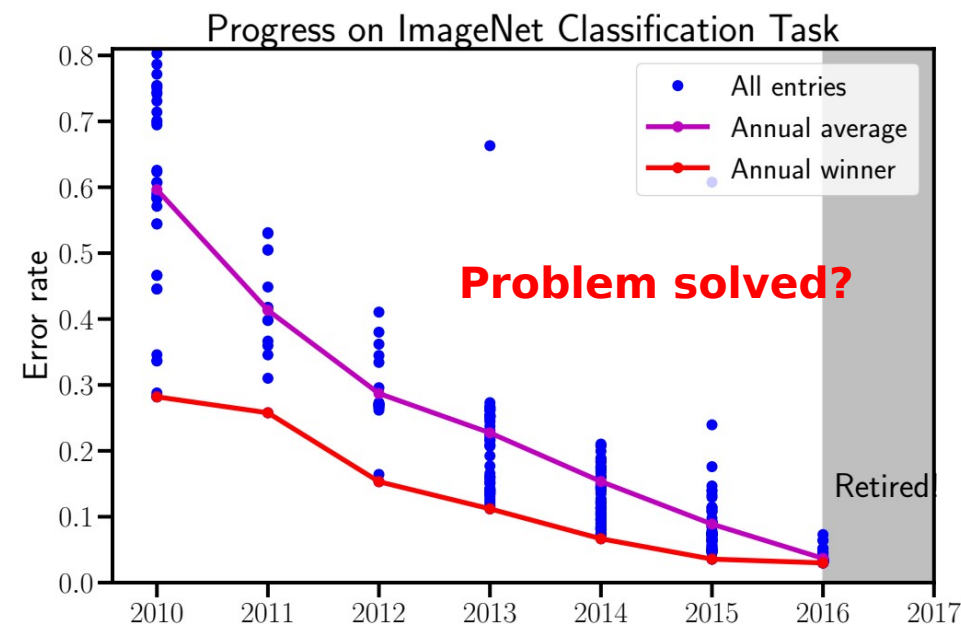
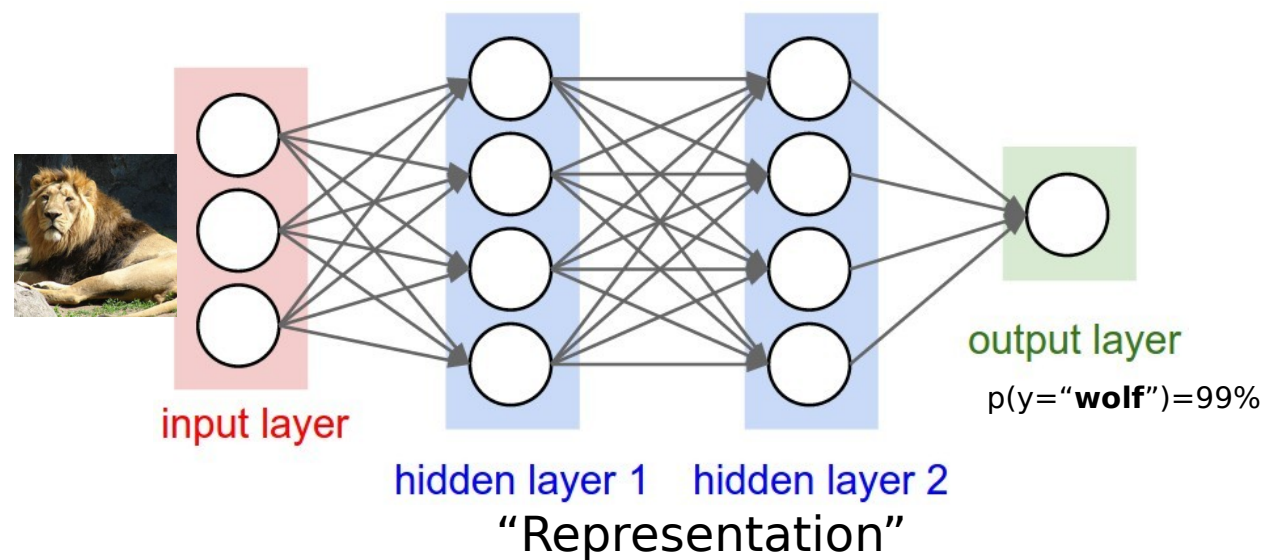
Another issue – far Out Of Distribution (OOD) inputs

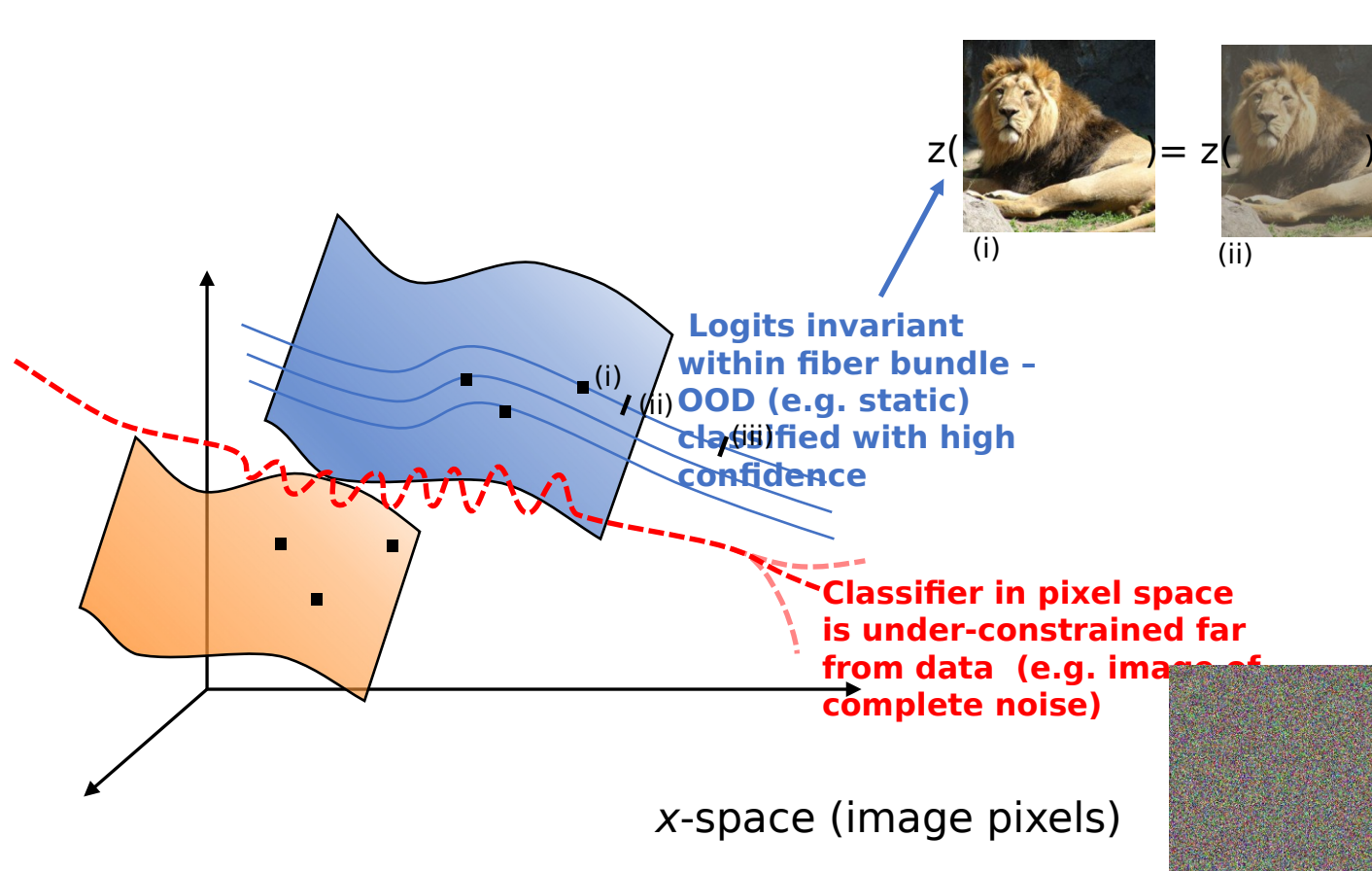


$p(y=\text{"lion"})=99\%$

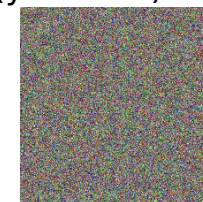


$p(y=\text{"lion"})=99\%$



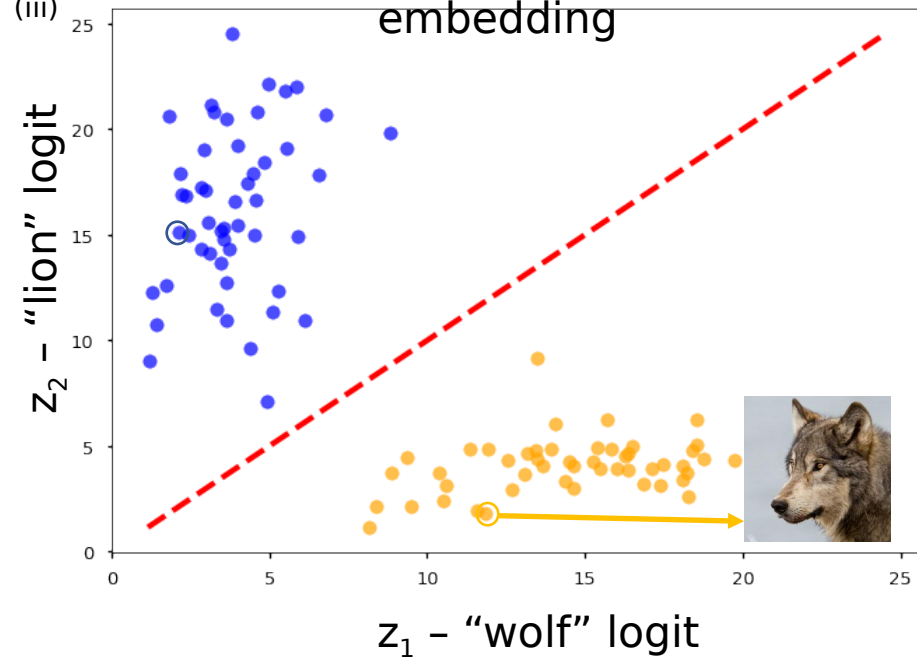


$p(y=\text{"lion"}) > 99\%$

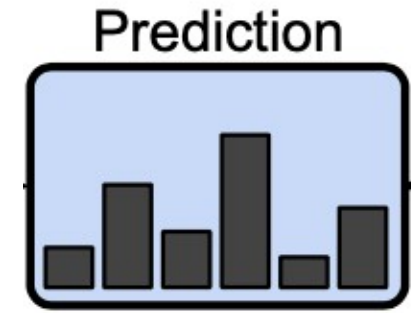


(iii)

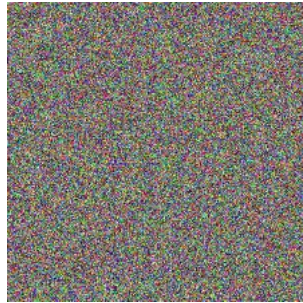
$z(x)$ - Logit embedding



What do the probabilities that our neural nets give us really *mean*?



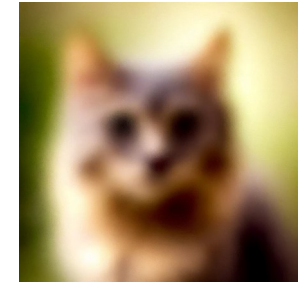
$p(y=\text{"lion"})=99\%$



$p(y=\text{"lion"})=99\%$



$p(y=\text{"lion"})=99\%$



$p(y=\text{"cat"})=80\%$

We often say that the probability a neural net assigns to the top choice is its **“confidence”**. If a neural net is 80% confident of a prediction, will that prediction be correct 80% of the time?

Confidence calibration game: give a range that you think the answer lies in with probability 50%

Martin Luther King's age at death:

Length of the Nile River:

Number of countries that are members of OPEC:

Number of books in the Old Testament:

My guesses

Correct answers

Martin Luther King's age at death:

35-55

39

Length of the Nile River:

700-1500 miles

6737 km or 4187 miles

Number of countries that are members of OPEC:

9-23

12 (as of 2026)

Number of books in the Old Testament:

30-55

39

Primer on Uncertainty & Robustness

Many of my slides on this are from
Balaji Lakshminarayanan

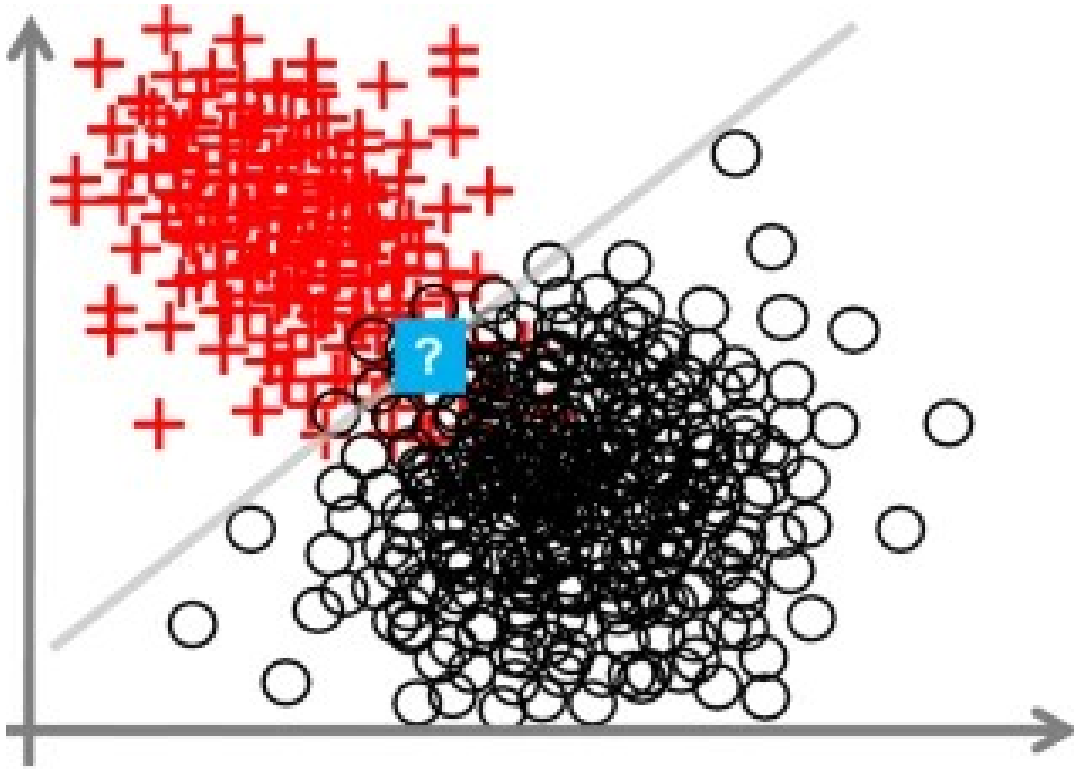
You can tell which ones because of the logo:



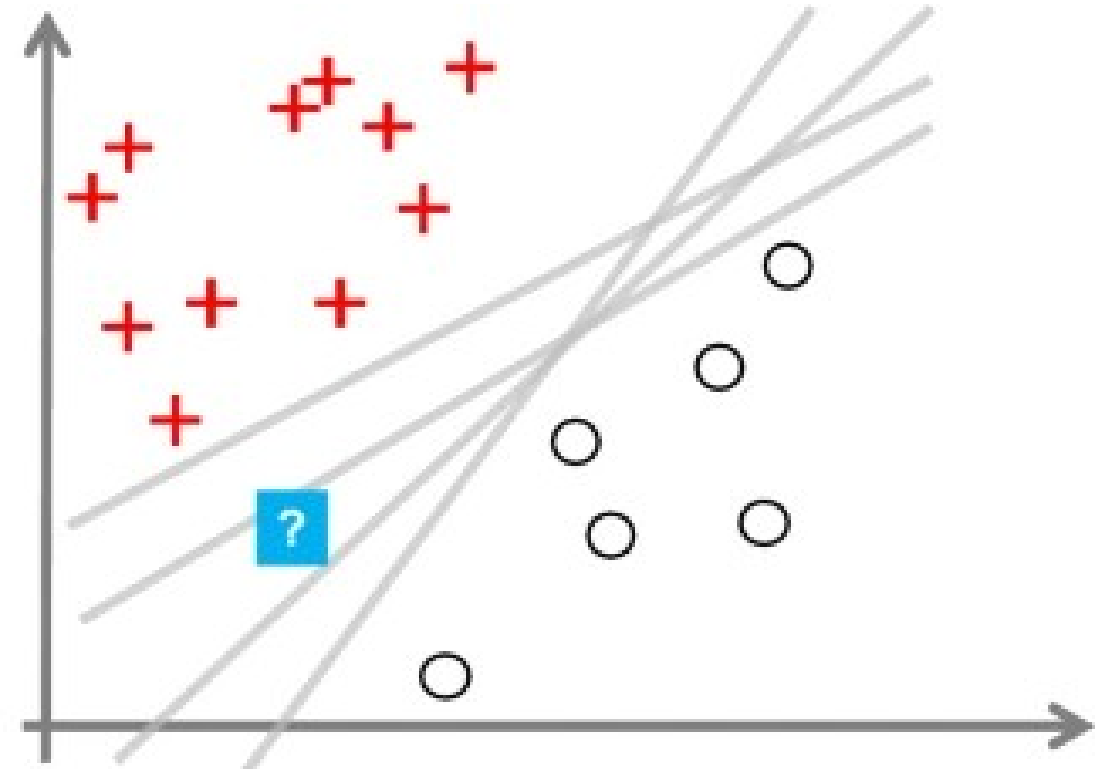
Video:

<https://slideslive.com/38935801/practical-uncertainty-estimation-outofdistribution-robustness-in-deep-learning>

Two types of uncertainty



aleatoric / fundamental / irreducible / data uncertainty

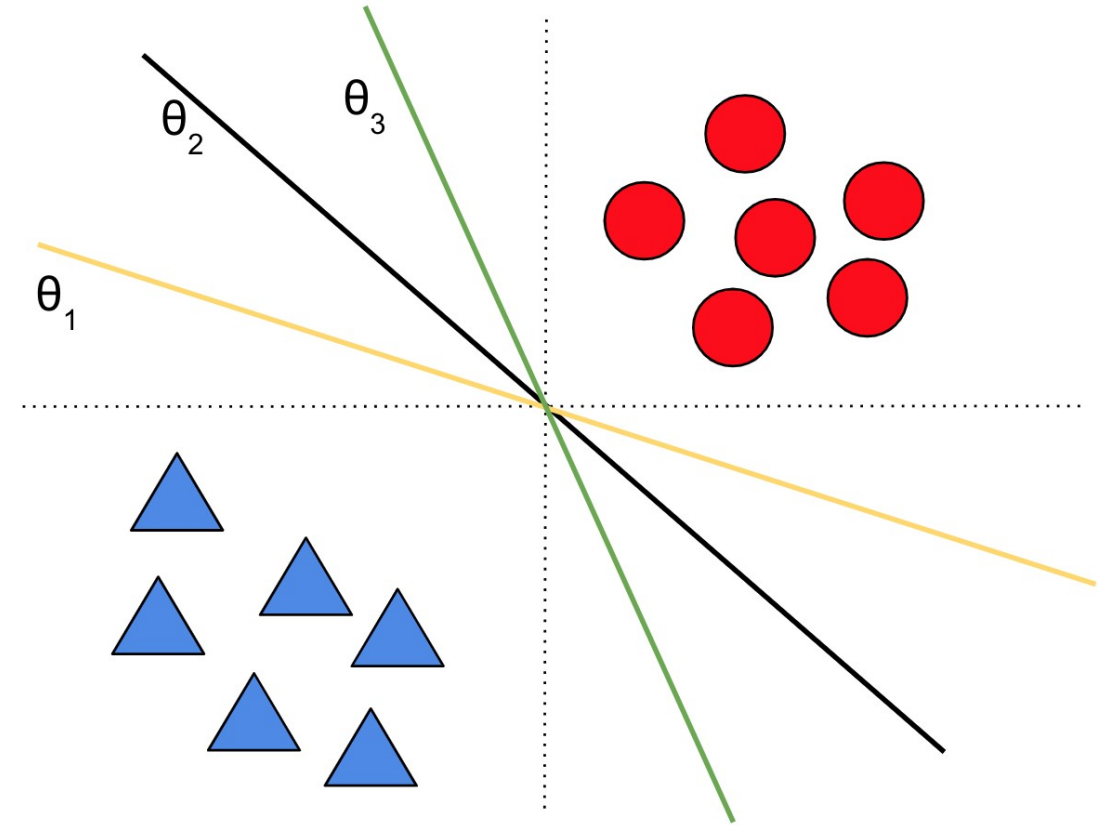


Epistemic / Model uncertainty
- due to our lack of knowledge

Do we care? Lots of applications. Right side will cause problems.

Sources of uncertainty: *Model uncertainty*

- Many models can fit the training data well
- Also known as ***epistemic uncertainty***
- Model uncertainty is “**reducible**”
 - Vanishes in the limit of infinite data (subject to model identifiability)



Sources of uncertainty: *Data uncertainty*

- Labeling noise (ex: human disagreement)

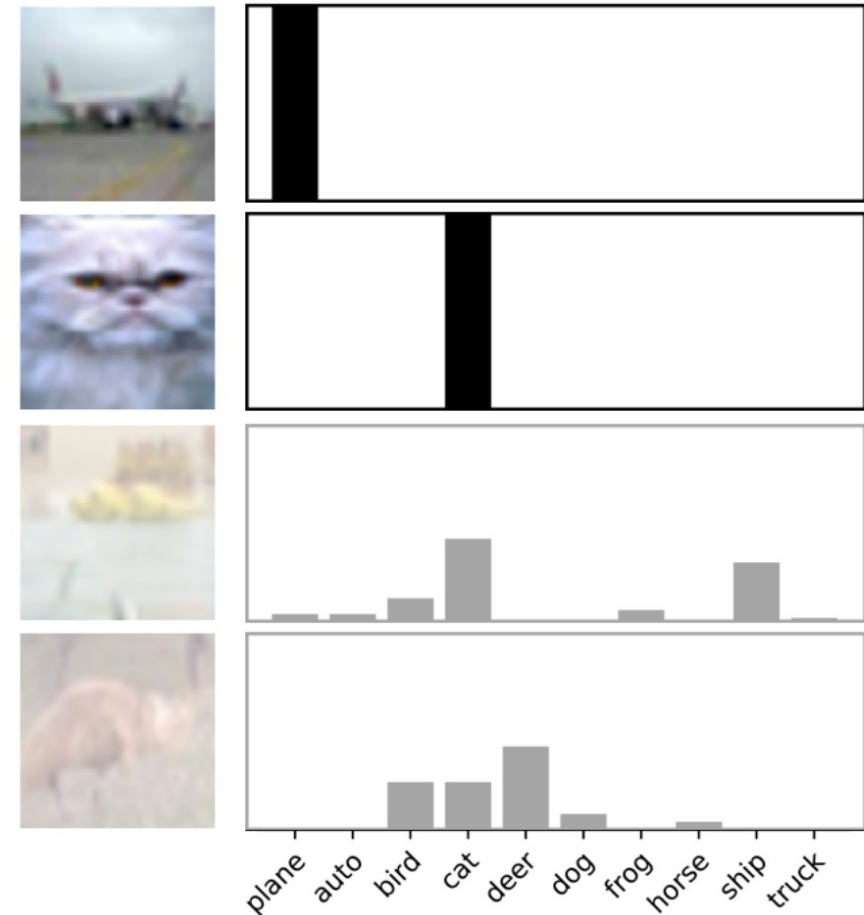


Image source: [Battleday et al. 2019](#) “Improving machine classification using human uncertainty measurements”

Sources of uncertainty: *Data uncertainty*

- Labeling noise (ex: human disagreement)

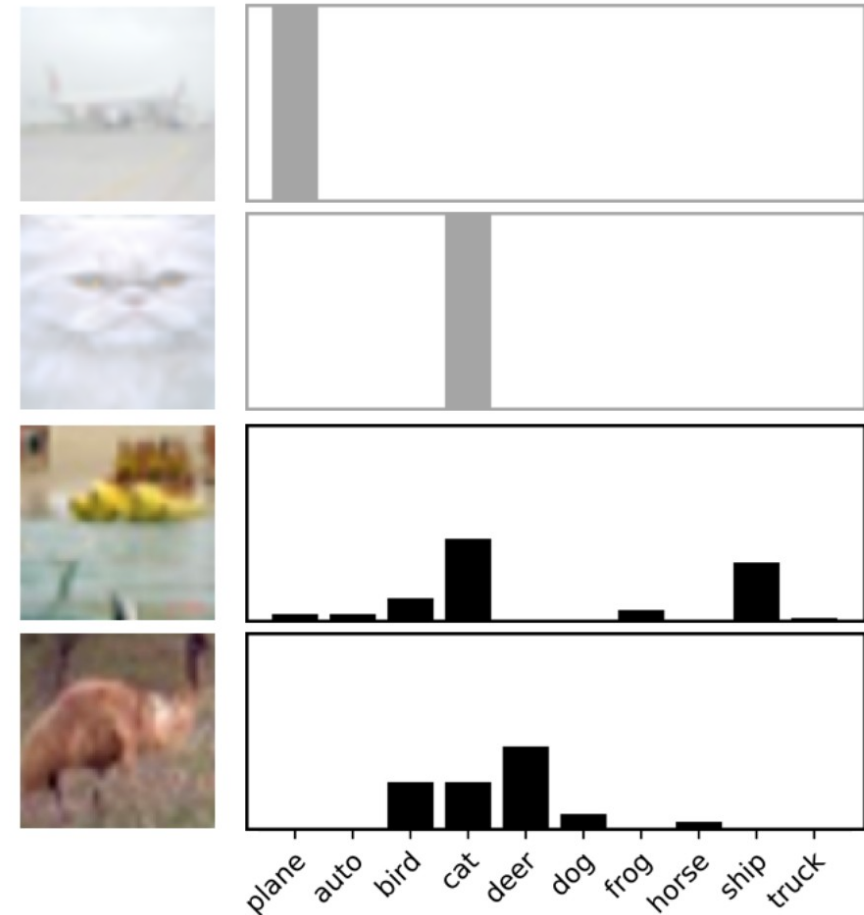


Image source: [Battleday et al. 2019](#) “Improving machine classification using human uncertainty measurements”

Sources of uncertainty: *Data uncertainty*

- Labeling noise (ex: human disagreement)
- Measurement noise (ex: imprecise tools)
- *Missing* data (ex: partially observed features, unobserved confounders)
- Also known as **aleatoric uncertainty**
- Data uncertainty is “**irreducible***”
 - Persists even in the limit of infinite data
 - *Could be reduced with additional

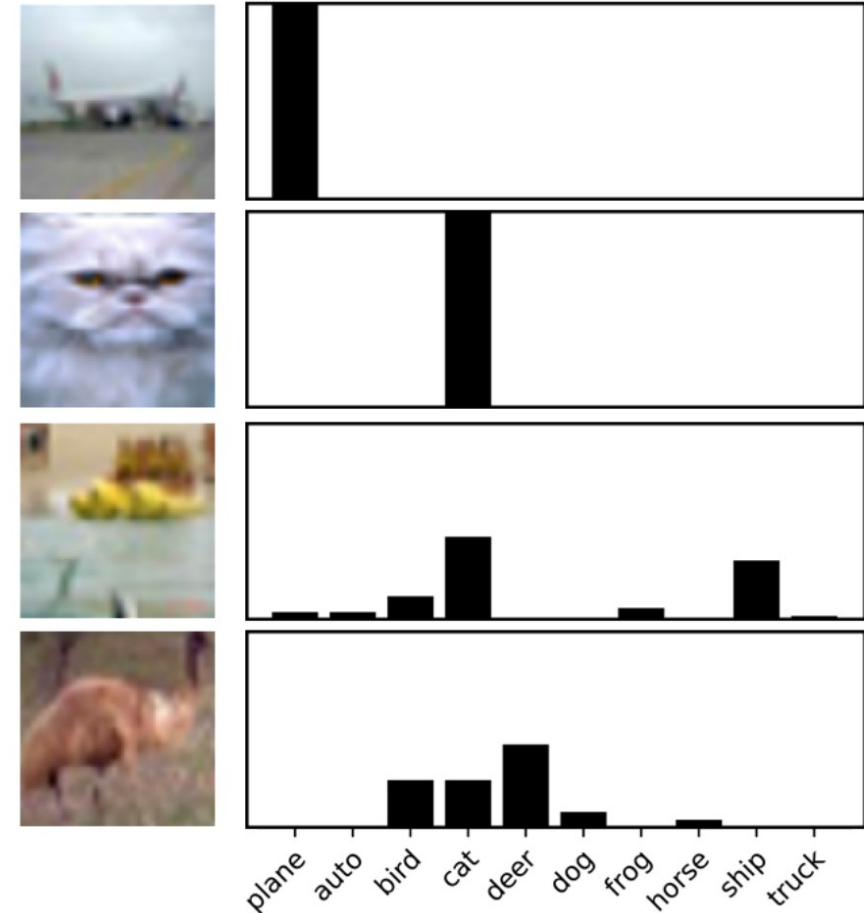


Image source: [Battleday et al. 2019](#) “Improving machine classification using human uncertainty measurements”

How do we measure the quality of uncertainty?

$$\text{Calibration Error} = |\text{Confidence} - \text{Accuracy}|$$

*predicted
probability of
correctness*

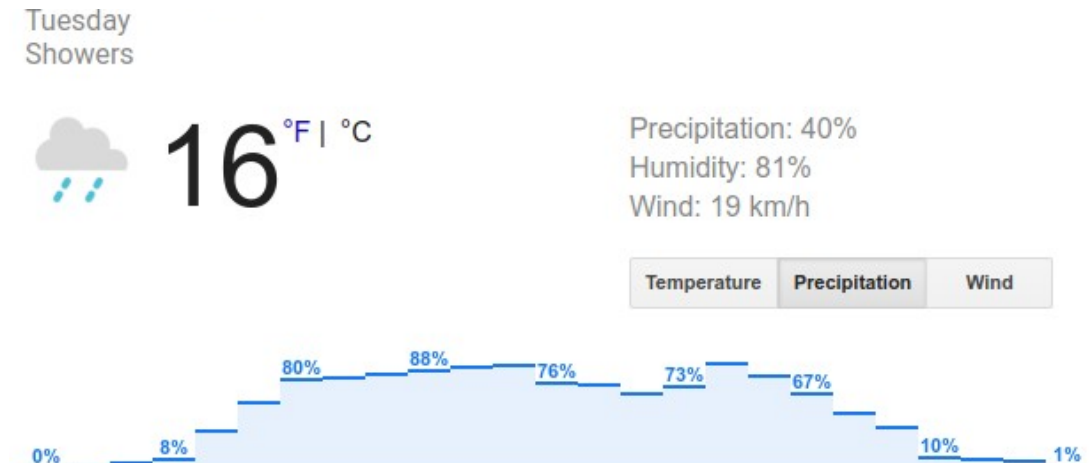
*observed
frequency of
correctness*

How do we measure the quality of uncertainty?

$$\text{Calibration Error} = |\text{Confidence} - \text{Accuracy}|$$

Of all the days where the model predicted rain with 80% probability, what fraction did we observe rain?

- 80% implies perfect calibration
- Less than 80% implies model is overconfident
- Greater than 80% implies model is underconfident



How do we measure the quality of uncertainty?

Expected Calibration Error [[Naeini+ 2015](#)]:

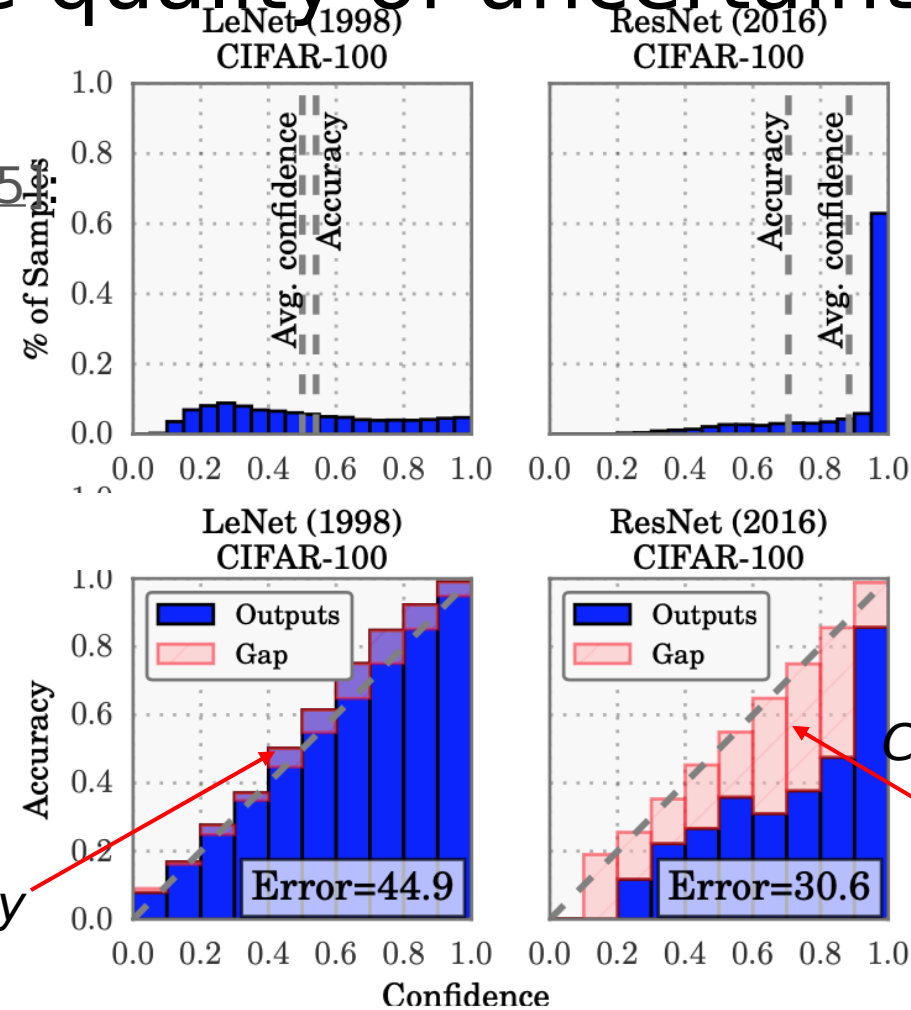
$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|$$

- Bin the probabilities into B bins.
- Compute the within-bin accuracy and within-bin predicted confidence.
- Average the calibration error across bins (weighted by number of points in each bin).

How do we measure the quality of uncertainty?

Expected Calibration Error [Naeini+ 2015]

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|$$



Confidence < Accuracy

=> Underconfident

Confidence > Accuracy

=> Overconfident

Image source: [Guo+ 2017](#) "On calibration of modern neural networks"

How do we measure the quality of uncertainty?

Expected Calibration Error [[Naeini+ 2015](#)]:

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|$$

*Note: Does **not** reflect **accuracy**.*

Predicting class frequency $p(y=1) = 0.3$ for all the inputs achieves perfect calibration.

[illegible]

How do we measure the quality of uncertainty?

Proper scoring rules [\[Gneiting & Raftery 2007\]](#)

- Negative Log-Likelihood (NLL)

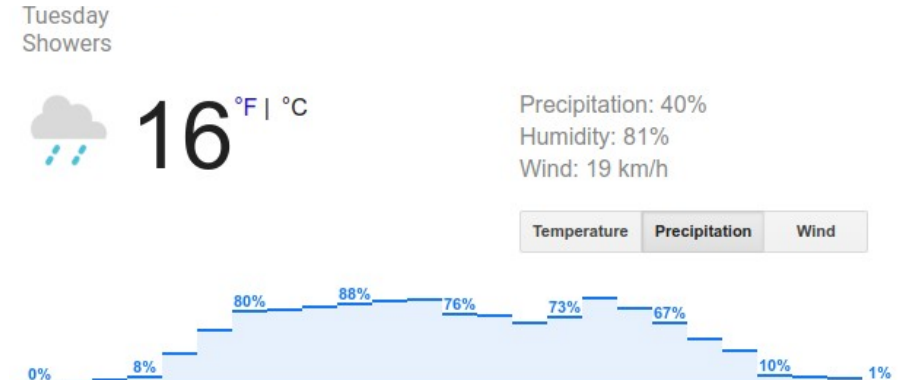
- Also known as *cross-entropy*
- Can overemphasize tail probabilities

- Brier Score

- Quadratic penalty (bounded range [0,1] unlike log).

$$BS = \frac{1}{|\mathcal{Y}|} \sum_{y \in \mathcal{Y}} [p(y|\mathbf{x}_n, \theta) - \delta(y - y_n)]^2$$

- Can be numerically unstable to optimize.



Simple Baselines

Simple Baseline: Recalibration

For classification, modify softmax probabilities **post-hoc**.

Temperature Scaling.

Parameterize output layer with scalar T .

$$p(y_i|x) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Minimize loss with respect to T on a separate “recalibration” dataset.

Caveat: Dataset shift...

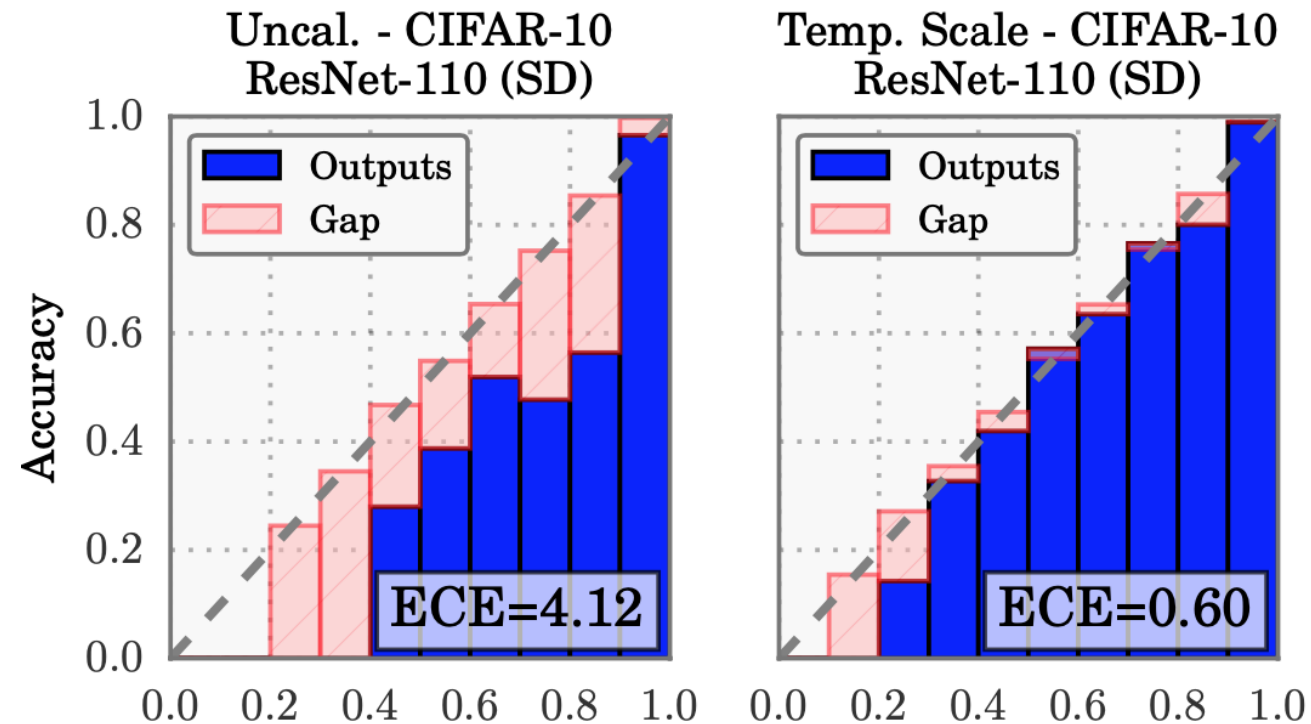
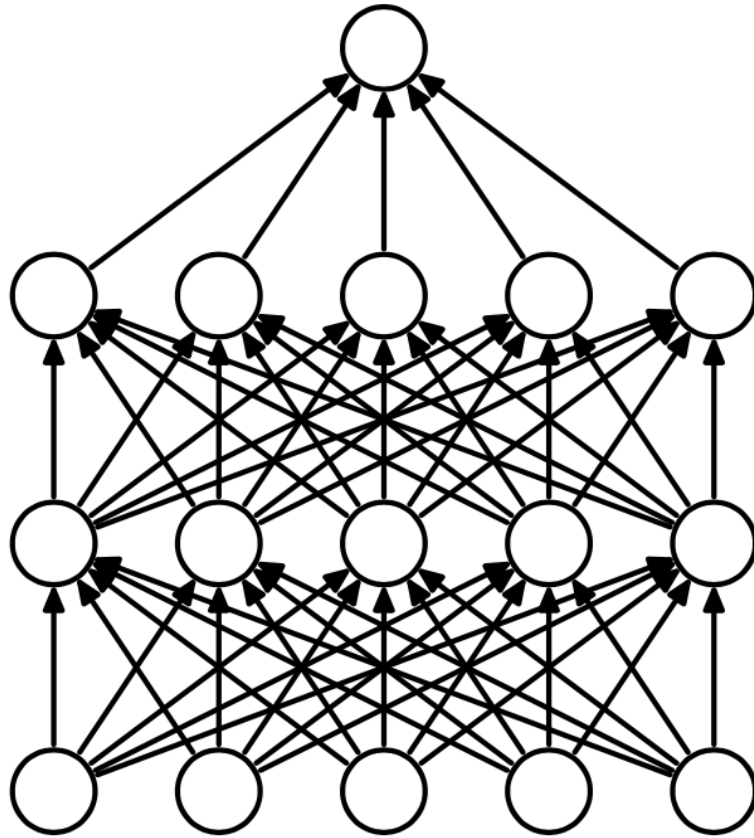
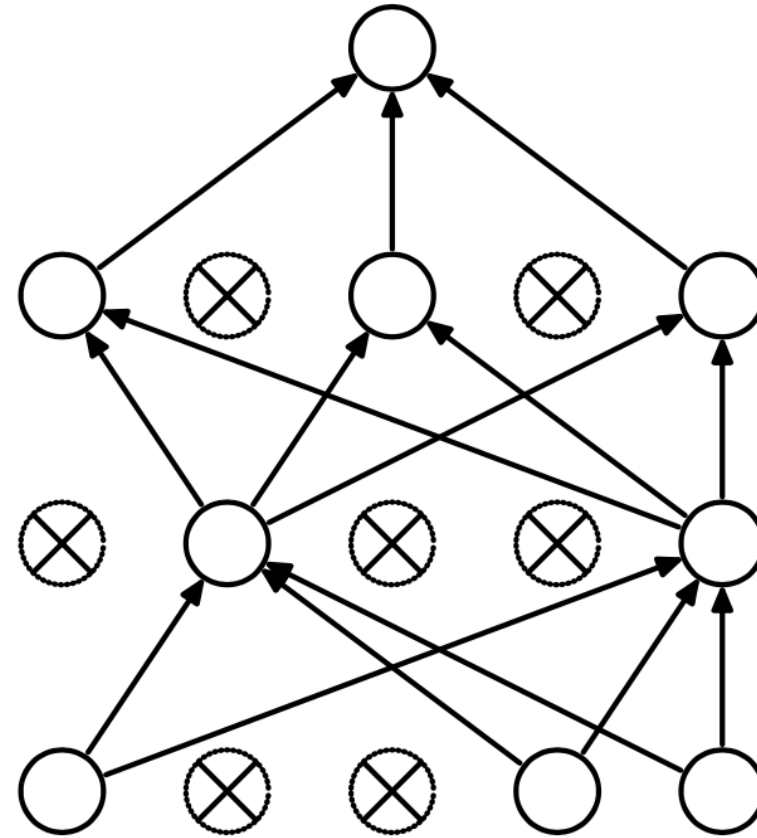


Image source: [Guo+ 2017](#) “On calibration of modern neural networks”

Simple Baseline: Monte Carlo Dropout



(a) Standard Neural Net



(b) After applying dropout.

Image source: Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Monte Carlo Dropout - average the probabilities

- Turn on dropout...

```
model.eval()
```

```
def enable_dropout(m):
```

```
    if isinstance(m, torch.nn.Dropout):
```

```
        m.train()
```

```
model.apply(enable_dropout)
```

Average the probabilities (pseudo-code):

```
avg_p = 0
```

```
for i in range(n_dropout):
```

```
    logits = model(x)
```

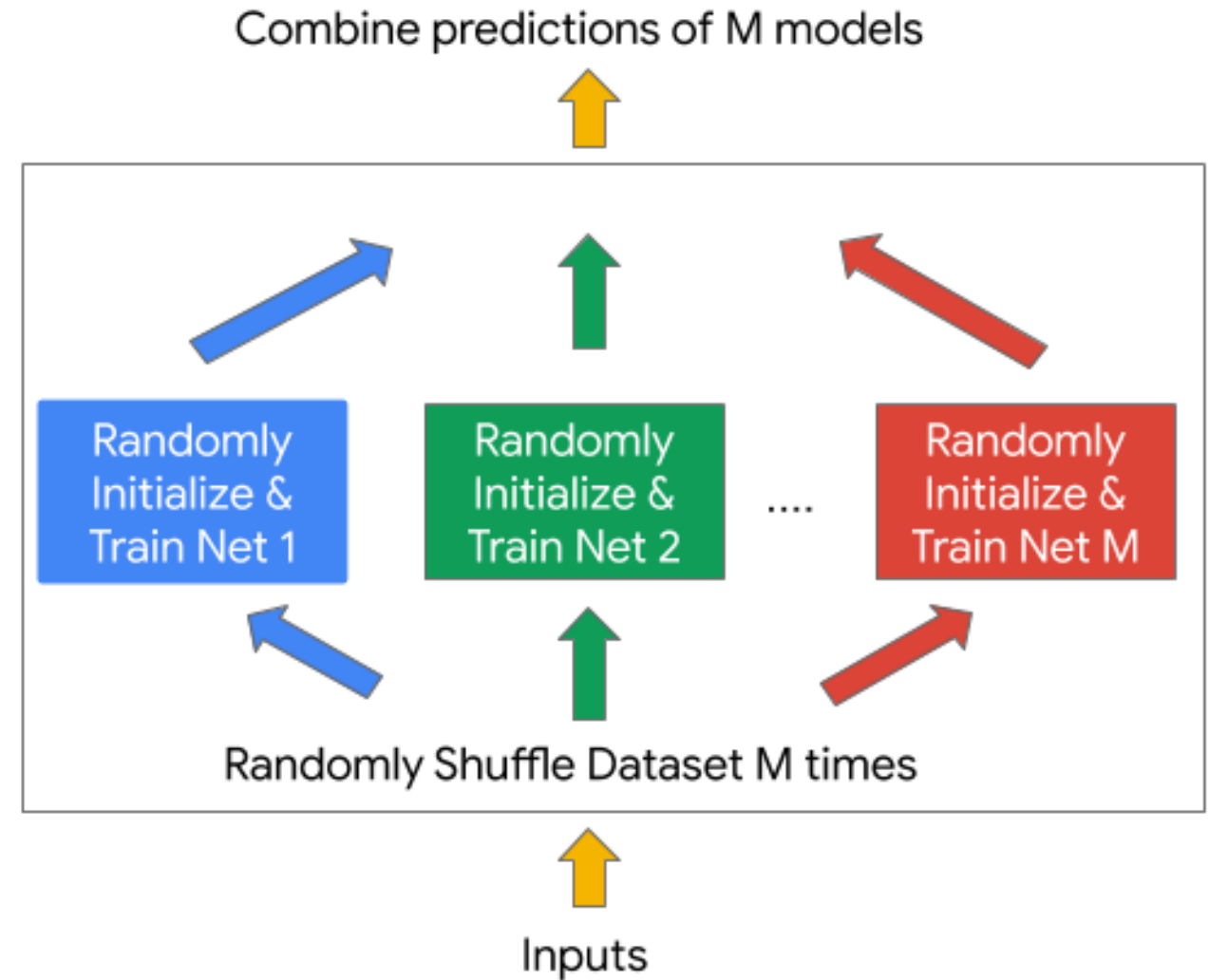
```
    p = softmax(logits)
```

```
    avg_p += p / n_dropout
```

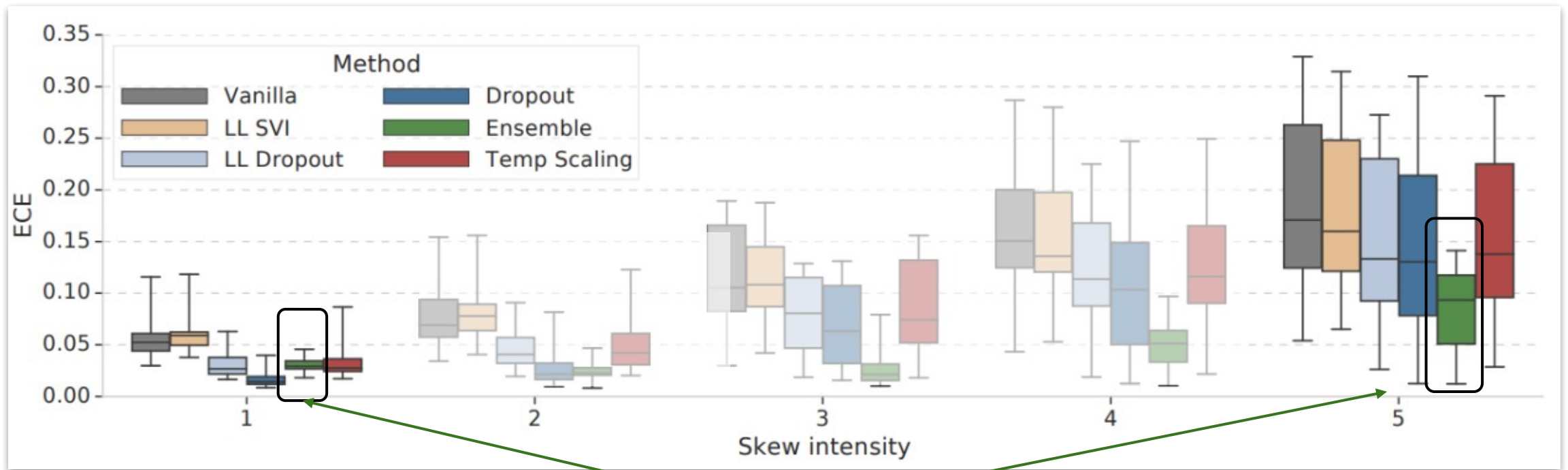
Simple Baseline: Deep Ensembles

Idea: Just re-run standard SGD training but with different random seeds and average the predictions

- A well known trick for getting better accuracy and Kaggle scores



Deep Ensembles work surprisingly well in practice

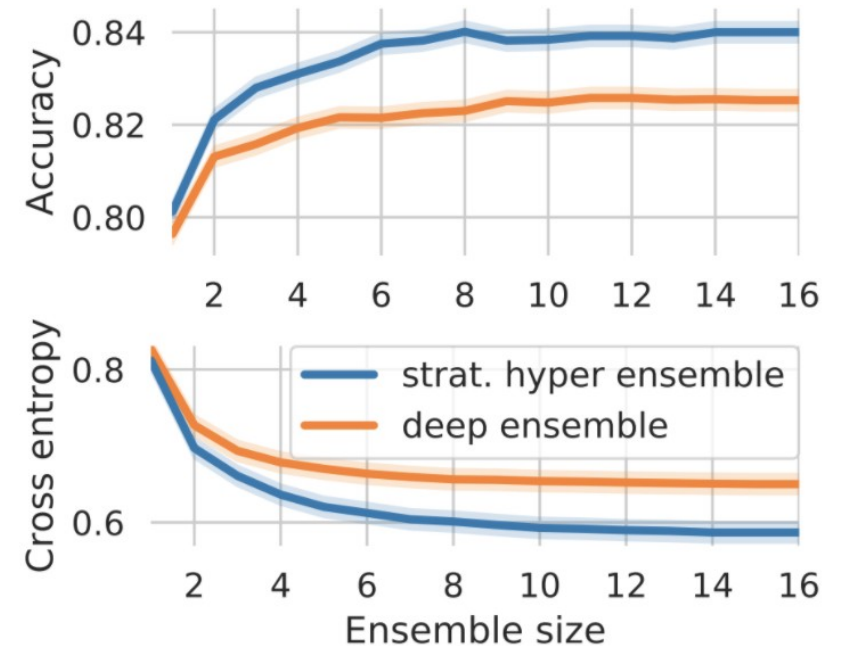


Deep Ensembles are consistently among the best performing methods, especially under dataset shift

Hyperparameter Ensembles

Deep ensembles differ only in random seed. By expanding the space of hyperparameters we average over, we can get even better accuracy & uncertainty estimates.

1. Run random search to generate a set of models.
 - a. Include random seed as part of the search space.
2. Greedily select the K models to pool.



Bayesian neural networks

- We'll get to these later in the class

Training to avoid over-confidence – label smoothing

$y = \text{cat}$

$X =$

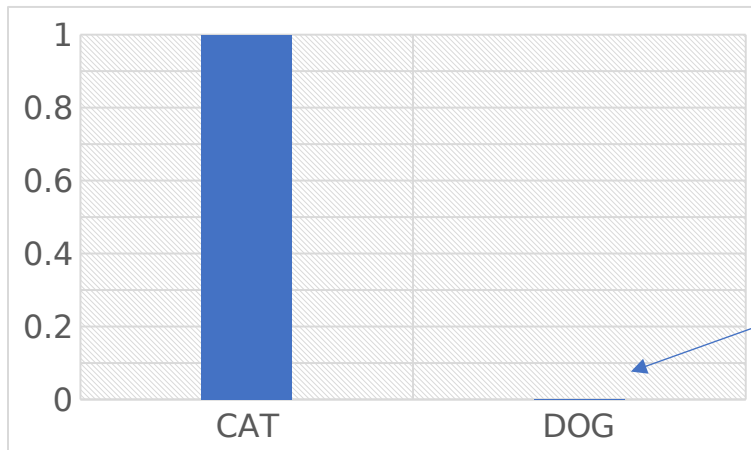


Show on board using largest marker possible

Super easy to try:

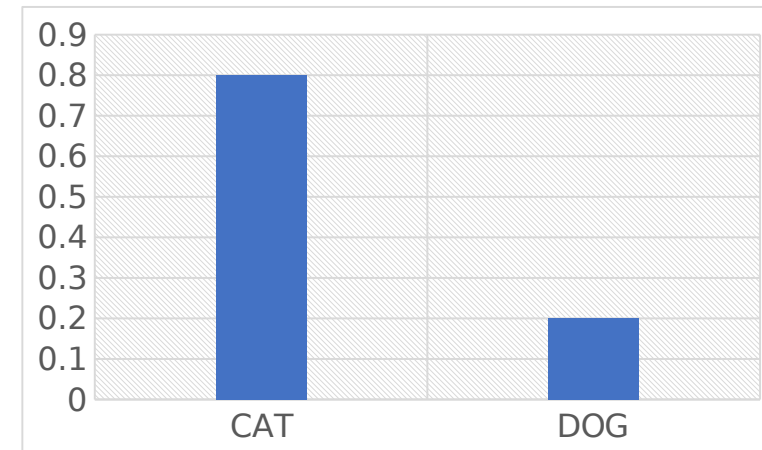
criterion =

`torch.nn.CrossEntropyLoss(label_smoothing=0.1)`



Label could be wrong?

$P_{\text{data}}(y|x)?$



$P_{\phi}(y|x)$

Applications of uncertainty quantification and domain shift

Known unknowns and unknown unknowns

I.I.D.

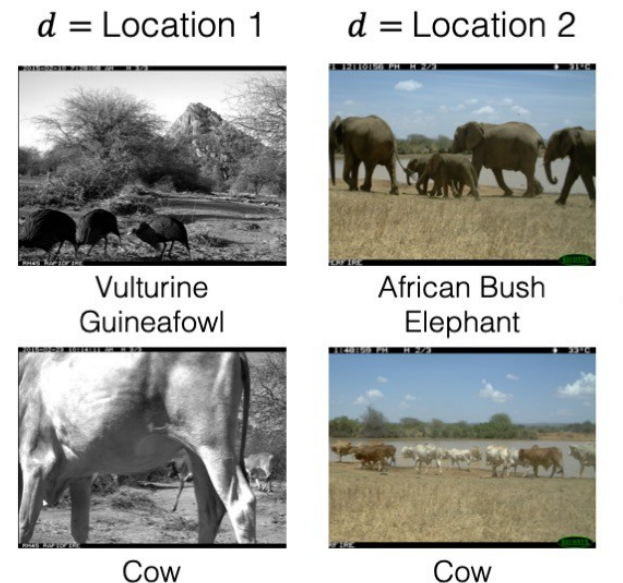
$$p_{\text{TEST}}(y, x) = p_{\text{TRAIN}}(y, x)$$

O.O.D.

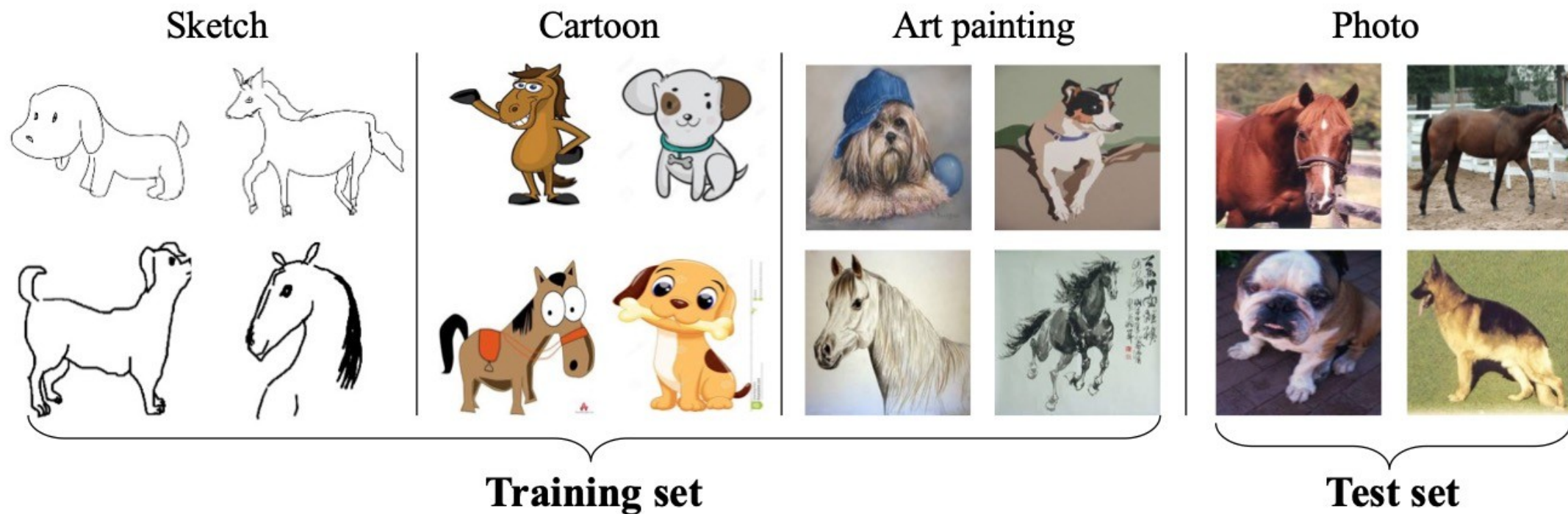
$$p_{\text{TEST}}(y, x) \neq p_{\text{TRAIN}}(y, x)$$

Examples of dataset shift:

- **Covariate shift.** Distribution of features $p(x)$ changes and $p(y|x)$ is fixed
- **Label shift.** Distribution of labels $p(y)$ changes and $p(x|y)$ is fixed

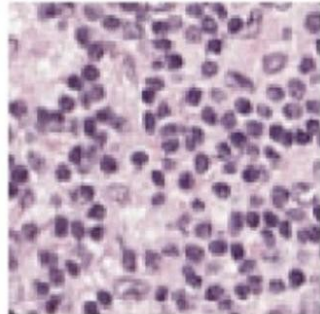
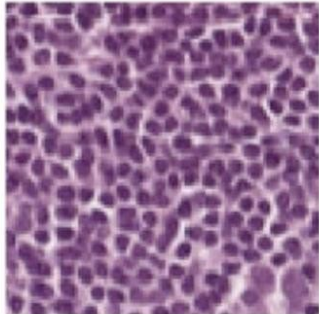
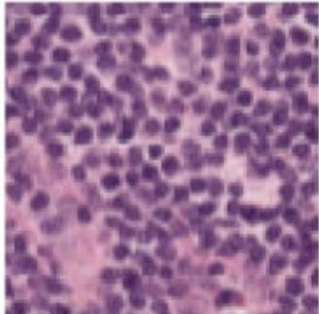
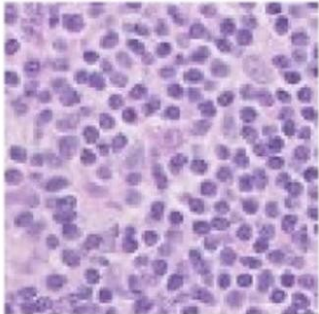
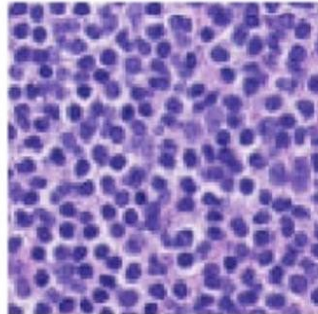
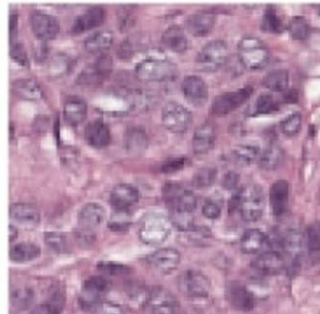
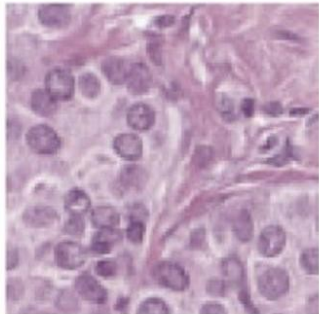
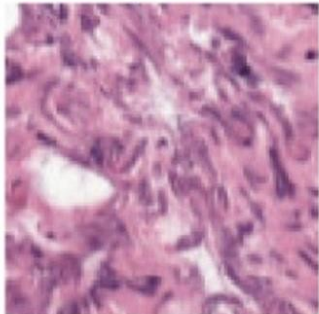
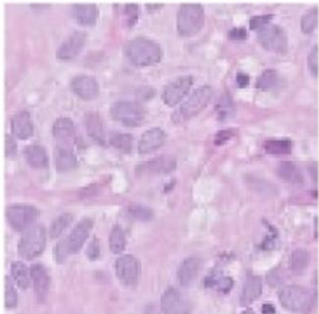
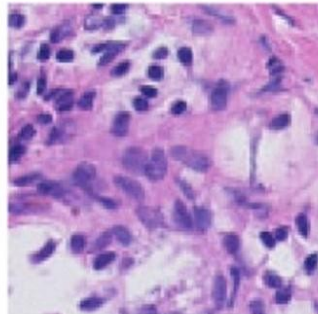


More extreme covariate shift



- $P_{Train}(x, y) \neq P_{Test}(x, y)$
- Source: Train, Target: test

Does the tissue contain tumor cells?

Train			Val (OOD)	Test (OOD)	
	d = Hospital 1	d = Hospital 2	d = Hospital 3	d = Hospital 4	d = Hospital 5
y = Normal					
y = Tumor					

95% accuracy, useful maybe! 70% accuracy in new hosp
Does the model *know* it is doing wor

Controllable experiments with dataset shift: ImageNet-C

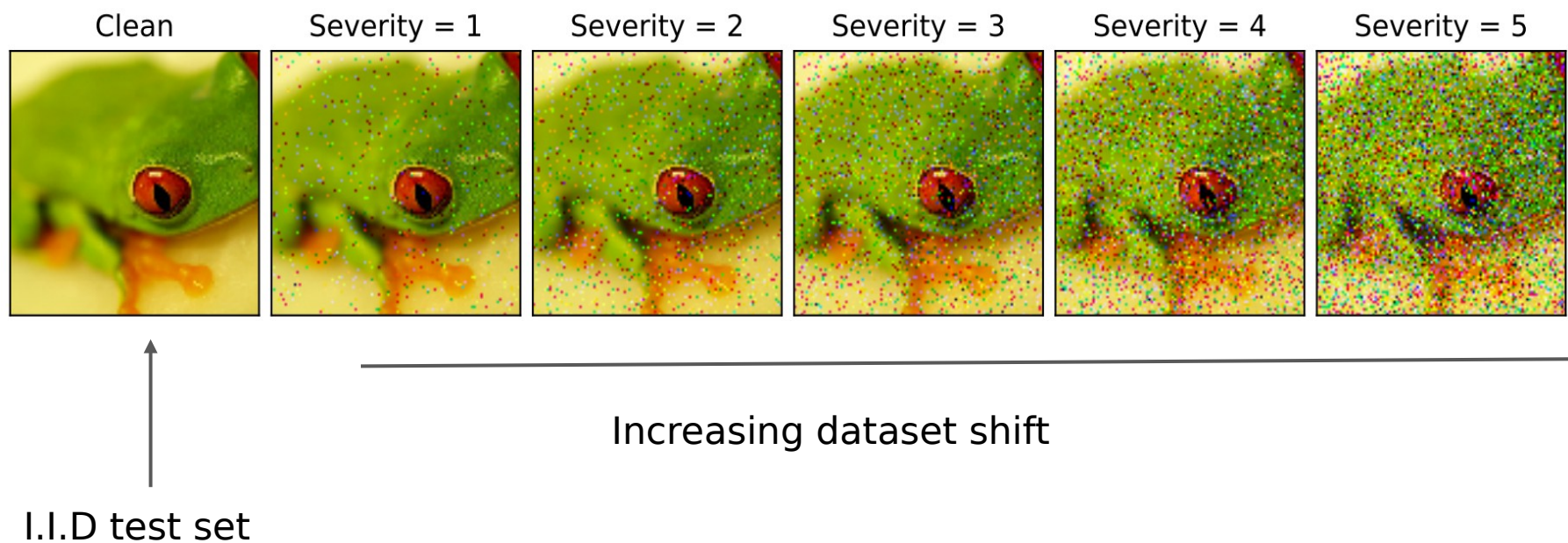


Image source: Benchmarking Neural Network Robustness to Common Corruptions and Perturbations, [Hendrycks & Dietterich, 2015](#)

ImageNet-C: Varying Intensity for Dataset Shift

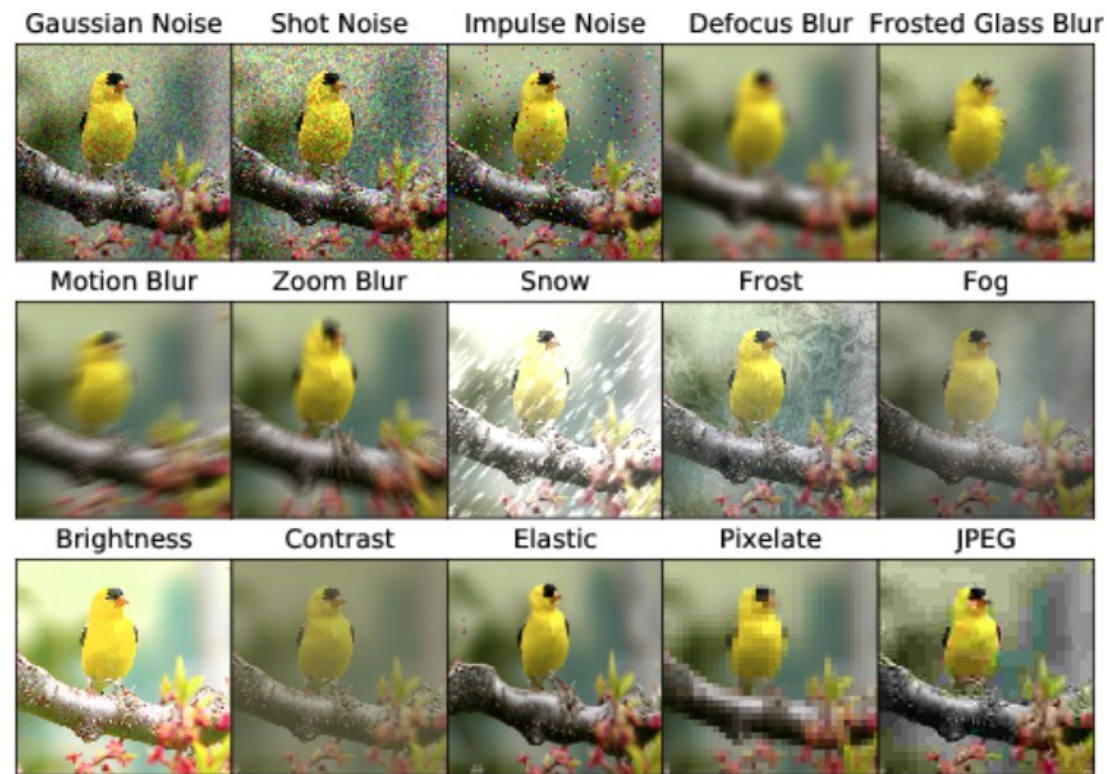
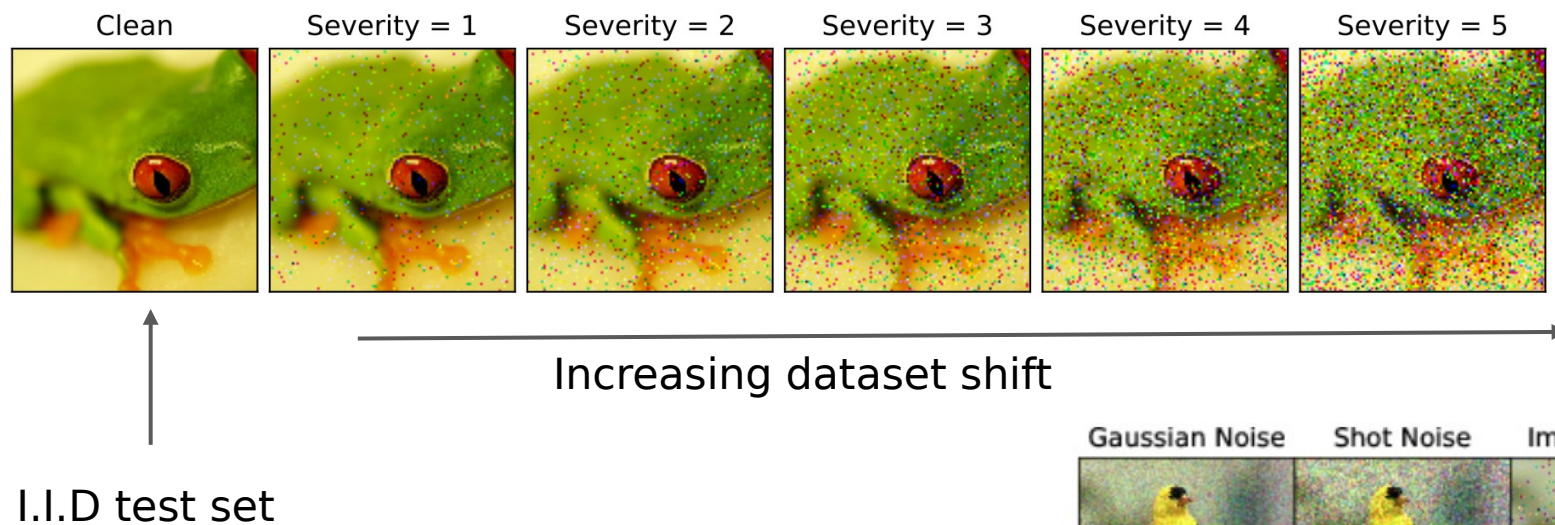
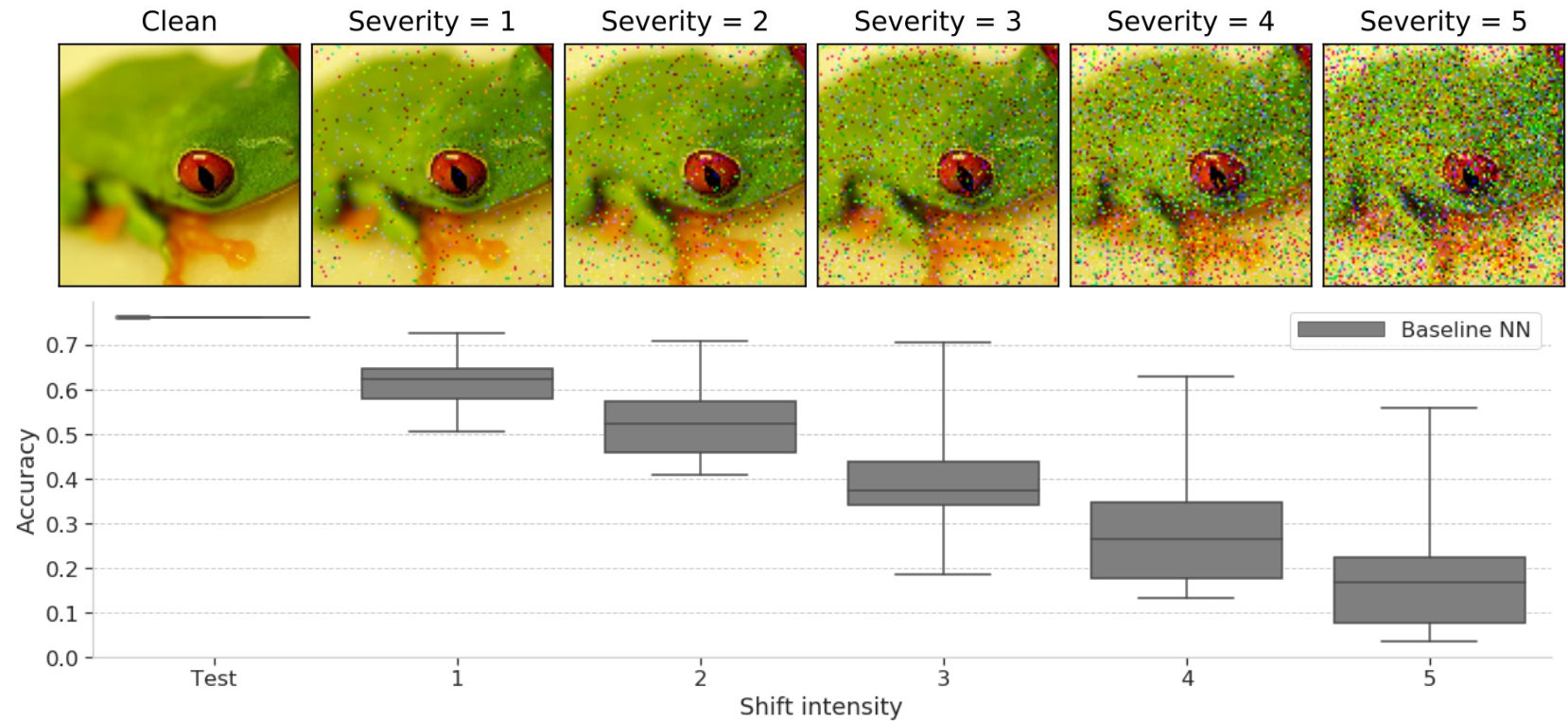


Image source: Benchmarking Neural Network Robustness to Common Corruptions and Perturbations, [Hendrycks & Dietterich, 2019](#).

Neural networks do not generalize under covariate shift

- **Accuracy drops** with increasing shift on Imagenet-C



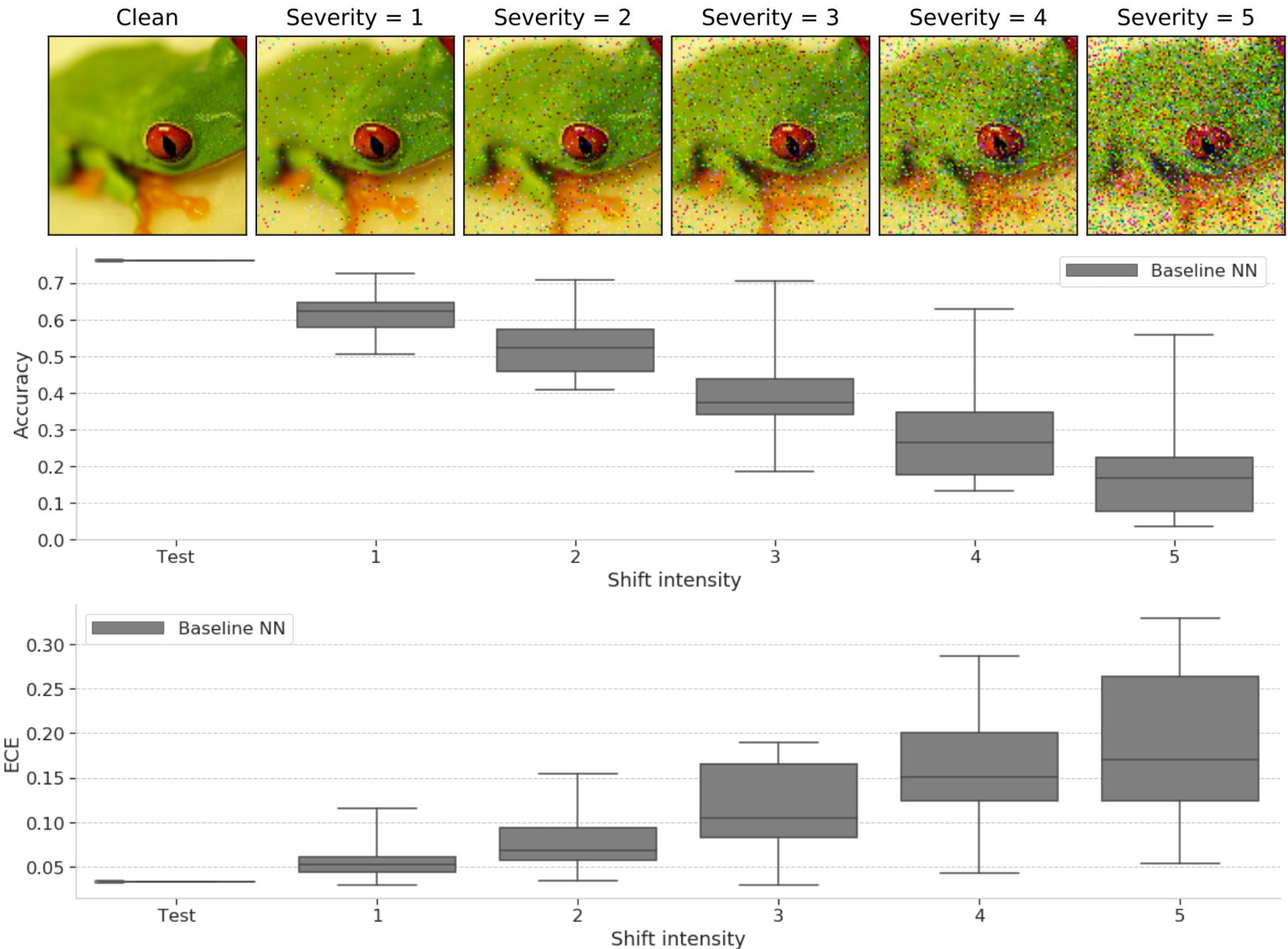
- But do the models know that they are less accurate?

Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift?, [Ovadia et al. 2019](#)

Neural networks *do not know when they don't know*

- **Accuracy drops** with increasing shift on Imagenet-C

- **Quality of uncertainty degrades with shift**
-> “overconfident mistakes”



Models assign high confidence predictions to OOD inputs

Example images where model assigns >99.5% confidence.

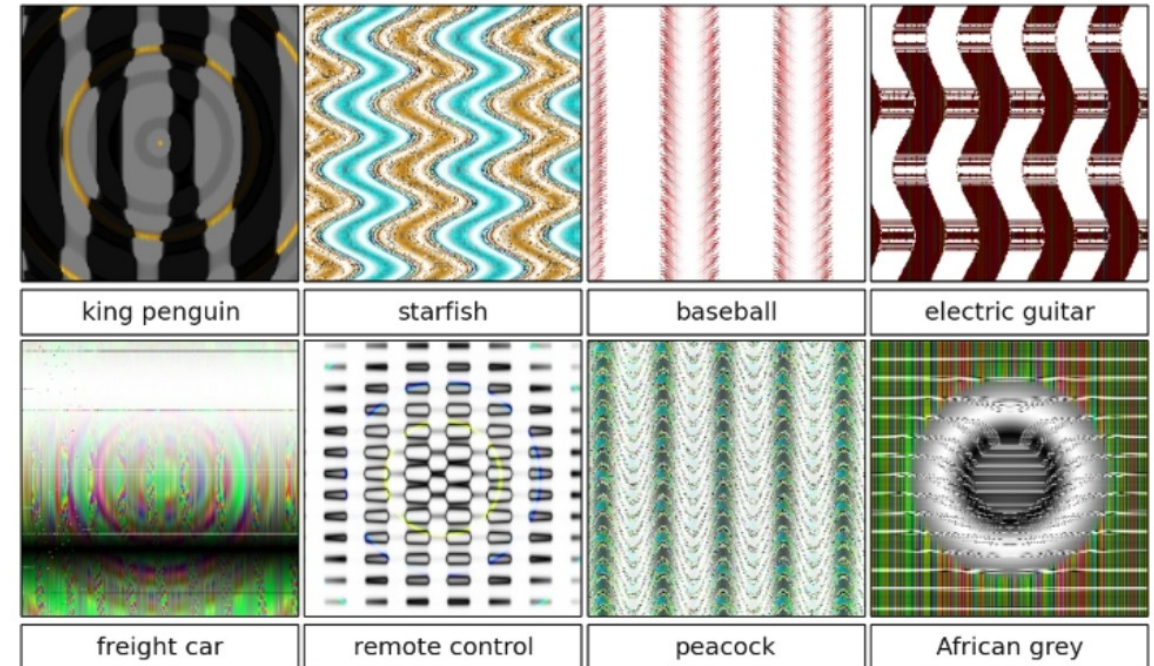
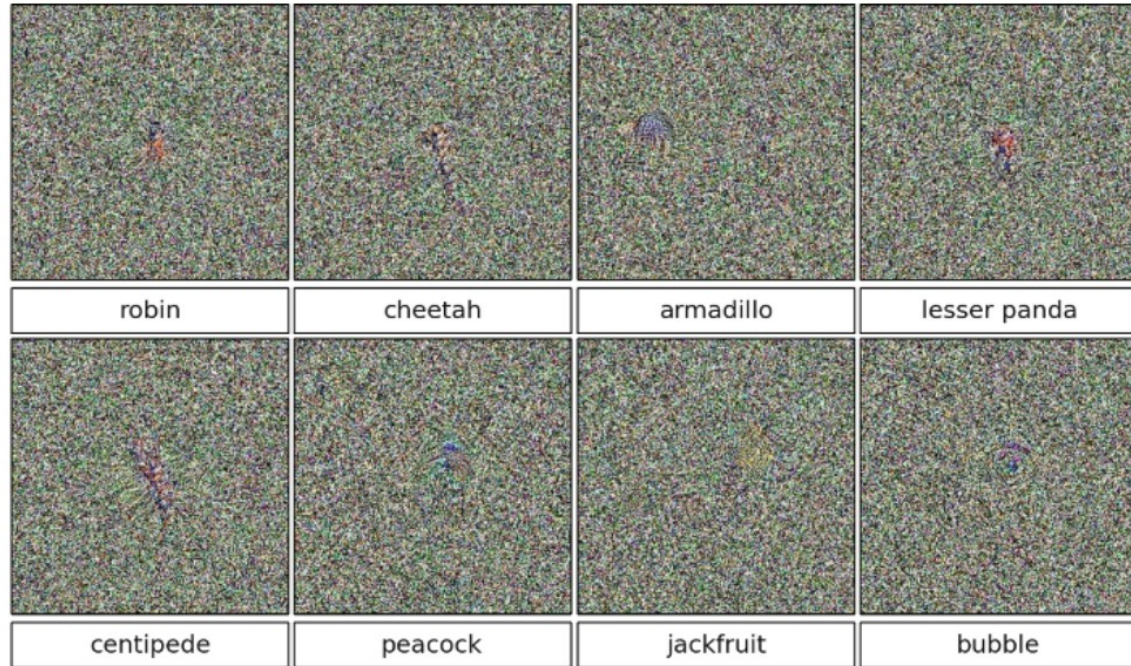


Image source: "Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images"
[Nguyen et al. 2014](#)

Models assign high confidence predictions to OOD inputs

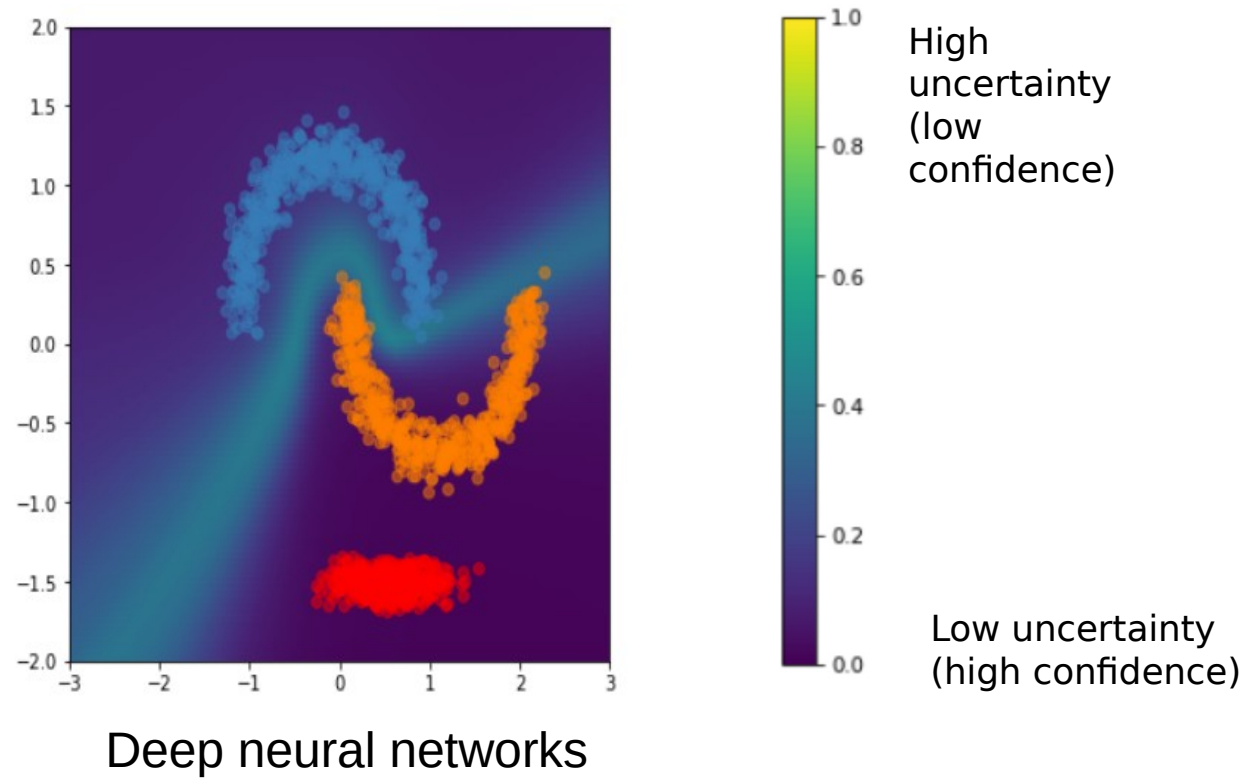
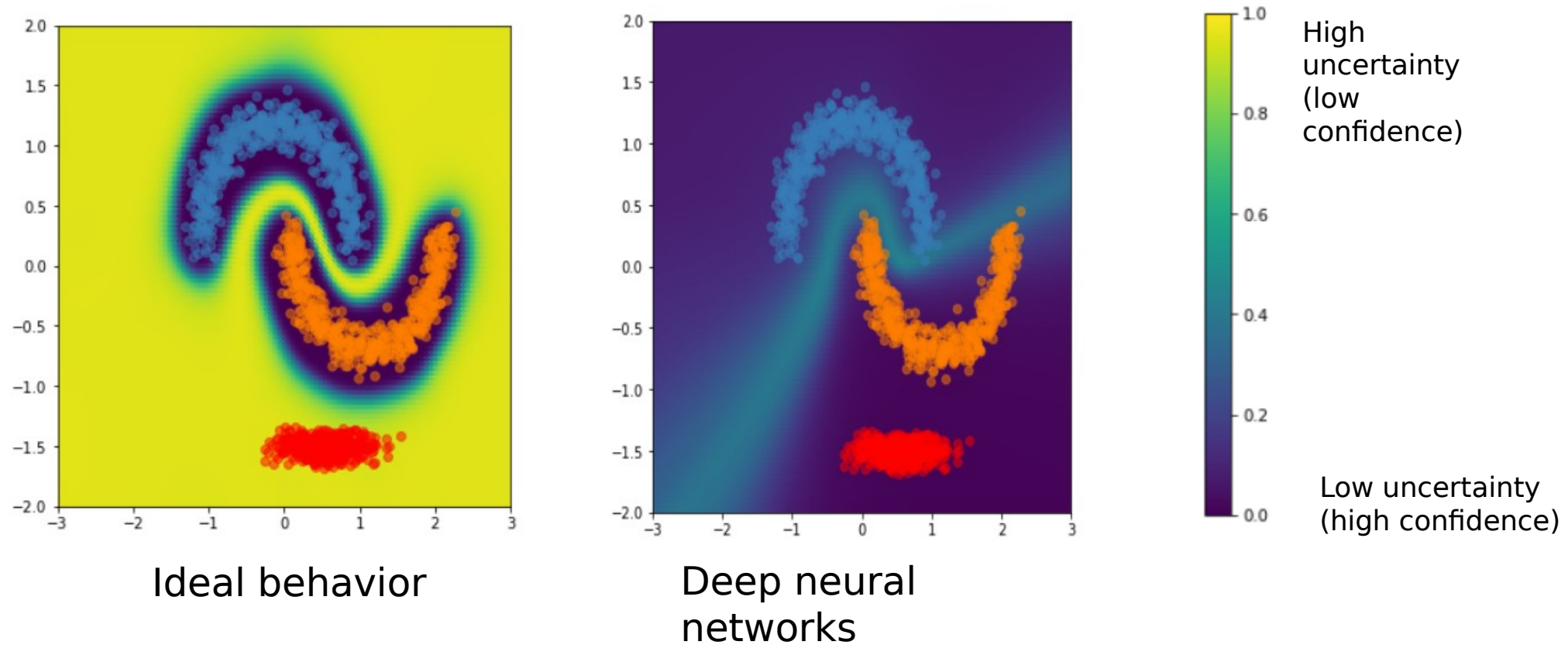


Image source: “Simple and Principled Uncertainty Estimation with Deterministic Deep Learning via Distance Awareness”

[Liu et al. 2020](#)

Models assign high confidence predictions to OOD inputs

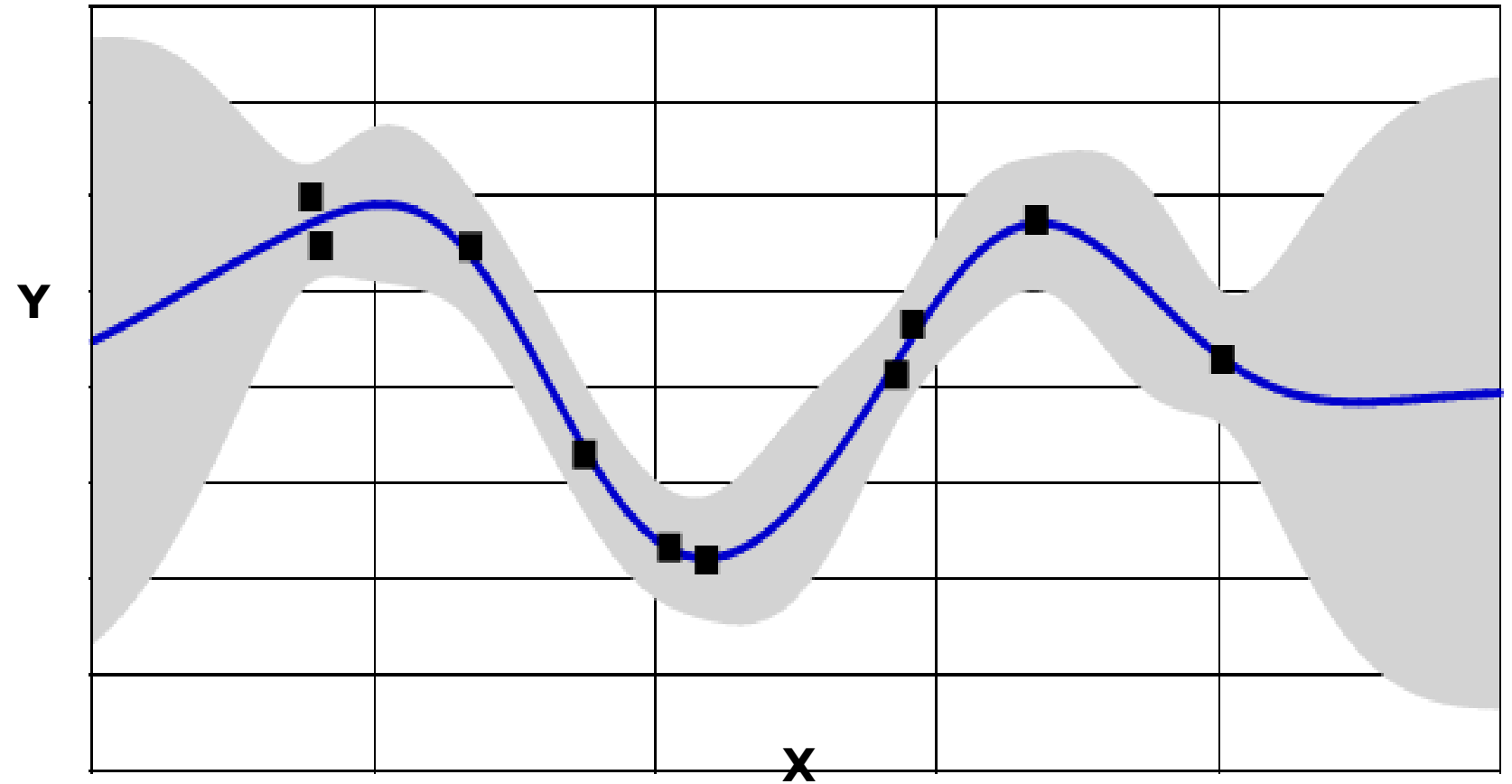


Trust model when x^* is close to $p_{\text{TRAIN}}(x,y)$

Image source: “Simple and Principled Uncertainty Estimation with Deterministic Deep Learning via Distance Awareness”

[Liu et al. 2020](#)

Aside: Uncertainty for regression

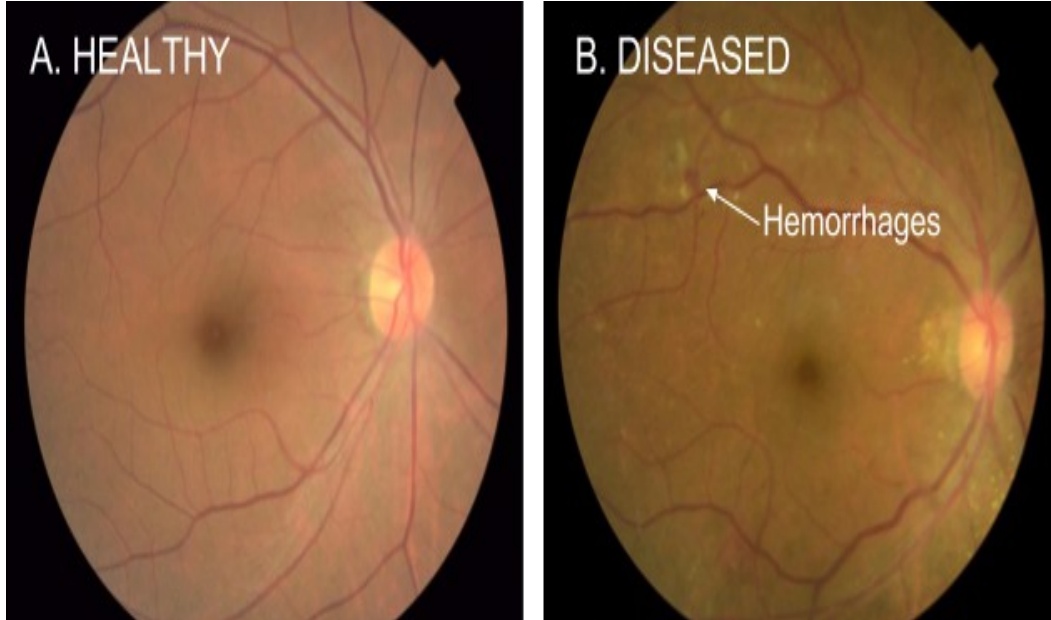


*Regression
(see “Conformal
Prediction” literature)*

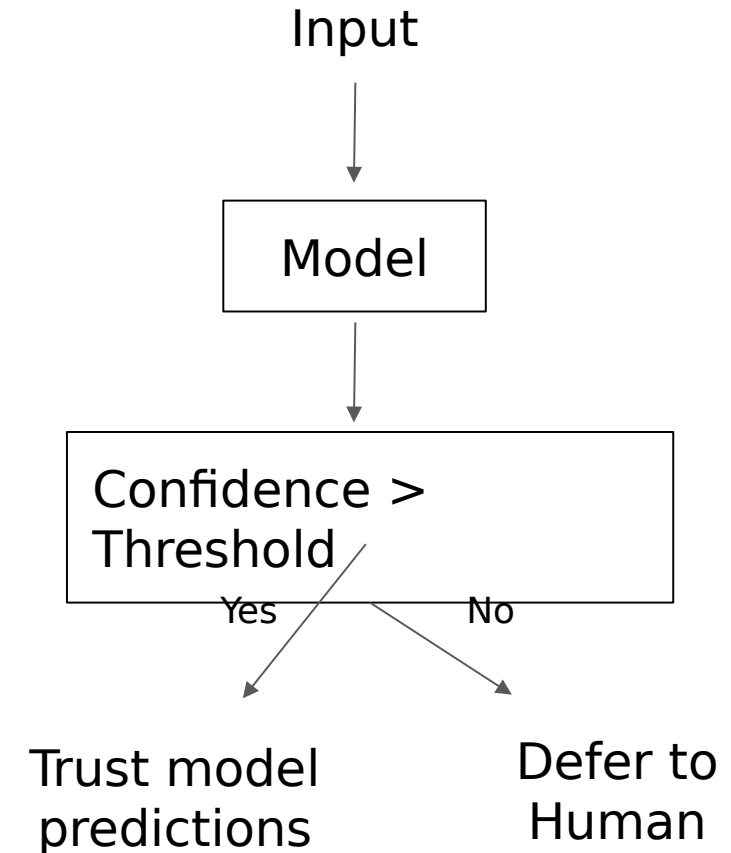
Applications

Healthcare

- Use model uncertainty to decide when to trust the model or to defer to a human.
- Reject low-quality inputs.



Diabetic retinopathy detection from fundus images [Gulshan et al, 2016](#)



Self-driving cars

Dataset shift:

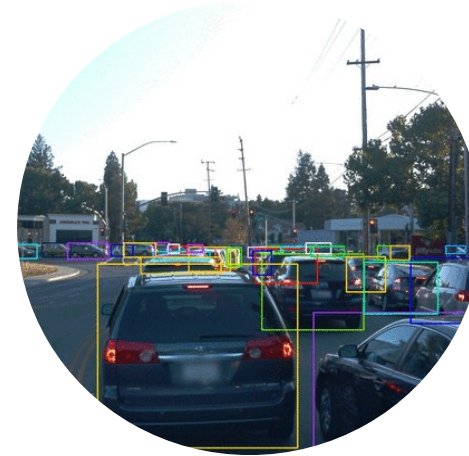
- Time of day / Lighting
- Geographical location (City vs suburban)
- Changing conditions (Weather / Construction)



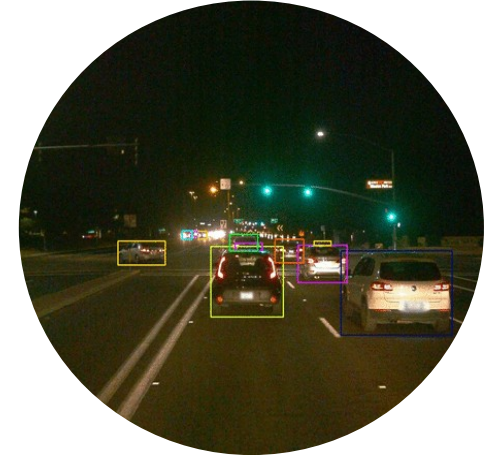
Weather



Construction



Daylight



Night



Downtown



Suburban

Image credit: Sun et al,
[Waymo Open Dataset](#)

Conversational Dialog systems

- Detecting out-of-scope utterances

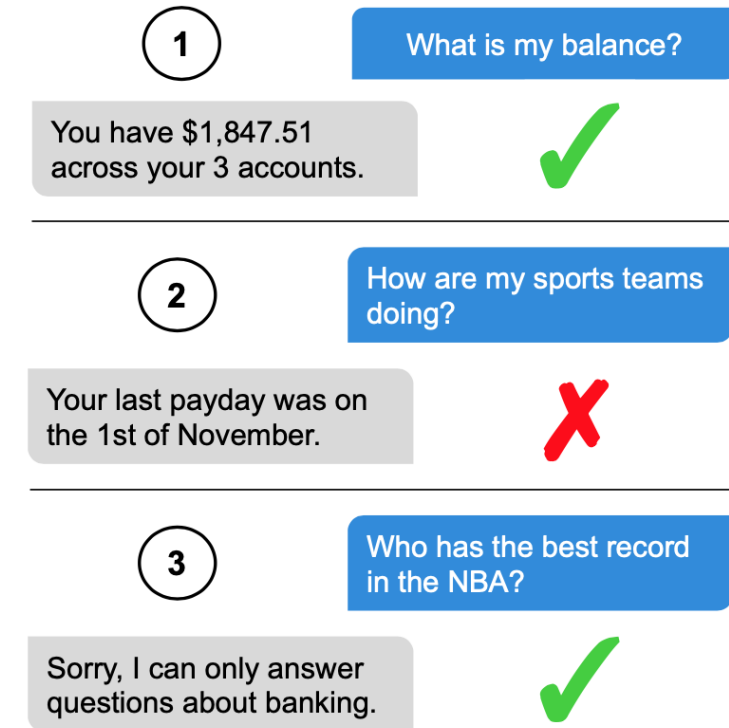


Figure 1: Example exchanges between a user (blue, right side) and a task-driven dialog system for personal finance (grey, left side). The system correctly identifies the user's query in ①, but in ② the user's query is mis-identified as in-scope, and the system gives an unrelated response. In ③ the user's query is correctly identified as out-of-scope and the system gives a fall-back response.

Image source: [Larson et al. 2019](#) "An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction"

Wrapping Up

Open Challenge: Scale

Enable uncertainty & robustness at the billion-trillion parameter scale.

Datasets. What is the role of priors on increasingly larger and diverse datasets?

Tasks. How do we think about OOD as we move toward general solutions to a wide range of tasks?

Model Parallelism. Mixtures of experts are already the backbone. Can we exploit recent ideas to enable even bigger and adaptive models?

Open Challenge: Benchmarks

In the past few years, there's been an ongoing call to action on benchmarks:

- *Comprehensive baselines* across standard and SOTA methods.
- *Large-scale models & datasets.*
- *High-quality code:* small changes in the setup can dramatically affect performance.

Preliminary efforts exist but a unified effort is required.

Today, we're happy to announce we made progress on this challenge.

Uncertainty Baselines

github.com/google/uncertainty-baselines

High-quality implementations of baselines on a variety of tasks.

Ready for use: 7 settings, including:

- Wide ResNet 28-10 on CIFAR
- ResNet-50 and EfficientNet on ImageNet
- BERT on Cline Intent Detection

14 different baseline methods.

Used across **10** projects at Google.

Collaboration with OATML @ Oxford, unifying
github.com/oatml/bdl-benchmarks.

dustinvtran and edward-bot Retune VI baseline for CIFAR. ...						
..						
README.md	Retune VI baseline for CIFAR.					3 hours ago
batchensemble.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
batchensemble_model.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
batchensemble_model_test.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
deterministic.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
deterministic_test.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
dropout.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
dropout_test.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
ensemble.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
utils.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago
variational_inference.py	Retune VI baseline for CIFAR.					3 hours ago
variational_inference_test.py	Move baselines/cifar10/ to baselines/cifar/.					13 days ago

README.md						
Wide ResNet 28-10 on CIFAR						
CIFAR-10						
Method	Train/Test NLL	Train/Test Accuracy	Train/Test Cal. Error	cNLL/cA/cCE	Train Runtime (hours)	# Parameters
Deterministic	1e-3 / 0.159	99.9% / 96.0%	1e-3 / 0.0231	1.29 / 69.8% / 0.173	1.2 (8 TPUv2 cores)	36.5M
BatchEnsemble (size=4)	0.08 / 0.143	99.9% / 96.2%	5e-5 / 0.0206	1.24 / 69.4% / 0.143	5.4 (8 TPUv2 cores)	36.6M
Dropout	2e-3 / 0.160	99.9% / 95.9%	2e-3 / 0.0241	1.35 / 67.8% / 0.178	1.2 (8 TPUv2 cores)	36.5M
Ensemble (size=4)	2e-3 / 0.114	99.9% / 96.6%	-	-	1.2 (32 TPUv2 cores)	146M
Variational inference	1e-3 / 0.211	99.9% / 94.7%	1e-3 / 0.029	1.46 / 71.3% / 0.181	5.5 (8 TPUv2 cores)	73M

Robustness Metrics

github.com/google-research/robustness_metrics

Lightweight modules to evaluate a model's robustness and uncertainty predictions.

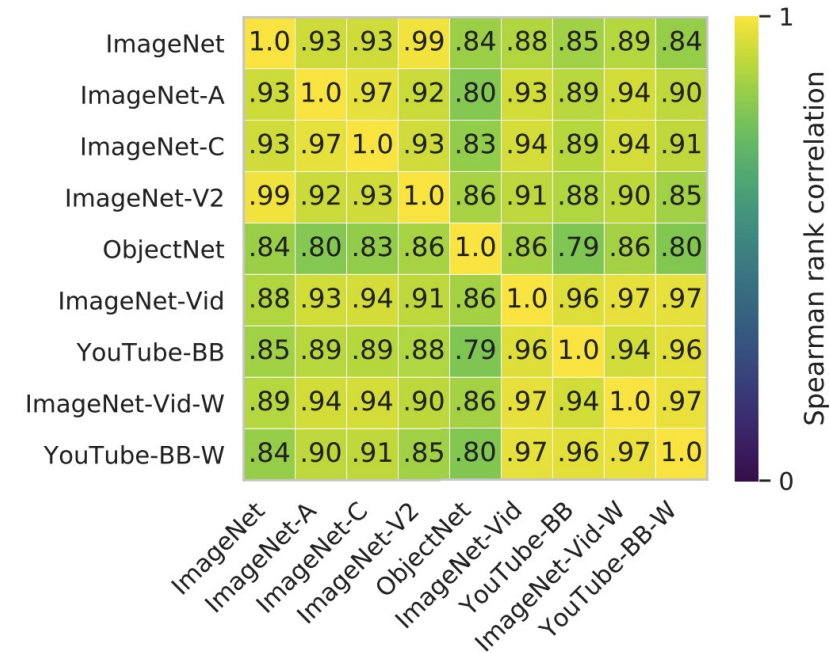
Ready for use:

- 10 OOD datasets
- Accuracy, uncertainty, and stability metrics
- Many SOTA models (TFHub support!)
- Multiple frameworks (JAX support!)

Enables large-scale studies of robustness

[[Djolonga+ 2020](#)].

Collaboration lead by Google Research, Brain Team @ Zurich.



Takeaways

- Uncertainty & robustness are critical problems in AI and machine learning.
- Benchmark models with calibration error and a large collection of OOD shifts.
- Probabilistic ML, ensemble learning, and optimization provide a foundation.
- The best methods advance two dimensions: combining multiple neural network predictions; and imposing priors and inductive biases.
- As compute increases, the uncertainty-robustness frontier outlines future progress.

My takeaways...

"Sophisticated" ML researchers think about uncertainty

There's no obvious universal approach

More success / hype on improving accuracy, compared to uncertainty estimates, but maybe that will change as GPT models saturate at human performance